



UNIVERZITET U NOVOM SADU
FAKULTET TEHNIČKIH NAUKA U
NOVOM SADU



Đorđe Šević

**Jedno rešenje za tokenizaciju platnog sistema
zatvorene petlje zasnovanog na blokčejn
tehnologiji**

DIPLOMSKI RAD

Novi Sad, 2024



УНИВЕРЗИТЕТ У НОВОМ САДУ & ФАКУЛТЕТ ТЕХНИЧКИХ НАУКА
21000 НОВИ САД, Трг Доситеја Обрадовића 6

КЉУЧНА ДОКУМЕНТАЦИЈСКА ИНФОРМАЦИЈА

Редни број, РБР:	
Идентификациони број, ИБР:	
Тип документације, ТД:	Монографска документација
Тип записа, ТЗ:	Текстуални штампани материјал
Врста рада, ВР:	Дипломски (Bachelor) рад
Аутор, АУ:	Ђорђе Шевић
Ментор, МН:	др Владимир Мандић, ванредни професор
Наслов рада, НР:	Једно решење за токенизацију платног система затворене петље заснованог на блокчејн технологији
Језик публикације, ЈП:	Српски језик
Језик извода, ЈИ:	Српски језик
Земља публикавања, ЗП:	Република Србија
Уже географско подручје, УП:	АП Војводина, Нови Сад
Година, ГО:	2024. година
Издавач, ИЗ:	Факултет техничких наука
Место и адреса, МА:	Трг Доситеја Обрадовића 6, 21 000 Нови Сад
Физички опис рада, ФО: (поглавља/страна/ цитата/табела/слика/графика/прилога)	8/60/51/3/21/0/14
Научна област, НО:	ИМТ – Информационе технологије
Научна дисциплина, НД:	Информационо – комуникациони системи (Инжењерство информационих система)
Предметна одредница/Кључне речи, ПО:	Затворен систем плаћања, токенизација, дигитални новчаник, блокчејн, микросервисна архитектура
УДК	
Чува се, ЧУ:	Библиотека Факултета техничких наука, Нови Сад
Важна напомена, ВН:	Нема
Извод, ИЗ:	Овај рад истражује имплементацију токенизације у системима плаћања затворене петље коришћењем блокчејн технологије, дигиталних новчаника и токена како би се побољшала сигурност трансакција и заштита података. Разматра се предности затворених система као што су контрола трошкова, лојалност корисника и смањење трансакционих трошкова. Предложено решење укључује токенизацију путем микроуслугне архитектуре, примену хеш функција и блокчејн технологије. Идентификоване предности су побољшана сигурност података, смањење ризика од повреде података и боља контрола приступа. Рад упоређује системе затворене и отворене петље, наглашавајући ограничења предложеног решења. Закључено је да интеграција токенизације и блокчејн технологије може побољшати сигурност и смањити трошкове трансакција. Даља унапређења, као што су уписивање доказа о трансакцијама у више блокчејнова и отворено банкарство, повећала би флексибилност и корисничко искуство.
Датум прихватања теме, ДП:	
Датум одбране, ДО:	13. 11. 2024.
Чланови комисије, КО:	Председник: др Ђорђе Пржуљ, редовни професор
	Члан: др Соња Ристић, редовни професор
	Члан, ментор: др Владимир Мандић, ванредни професор
	Потпис ментора



UNIVERSITY OF NOVI SAD & FACULTY OF TECHNICAL SCIENCES
21000 NOVI SAD, Trg Dositeja Obradovića 6

KEY WORDS DOCUMENTATION

Accession number, ANO :			
Identification number, INO :			
Document type, DT :	Monographic publication		
Type of record, TR :	Textual printed document		
Contents code, CC :	Bachelor thesis		
Author, AU :	Đorđe Šević		
Mentor, MN :	Vladimir Mandić, PhD, associate professor		
Title, TI :	One solution for the tokenization of a closed-loop payment system based on the blockchain technology		
Language of text, LT :	Serbian		
Language of abstract, LA :	Serbian		
Country of publication, CP :	Republic of Serbia		
Locality of publication, LP :	AP Vojvodina, Novi Sad		
Publication year, PY :	2024		
Publisher, PB :	Faculty of Tehnical Sciences		
Publication place, PP :	Trg Dositeja Obradovića 6, 21000 Novi Sad		
Physical description, PD : (chapters/pages/ref./tables/pictures/graphs/appendixes)	8/60/51/3/21/0/14		
Scientific field, SF :	IMT – Information Technologies		
Scientific discipline, SD :	Information and Communication Systems (Information Systems Engineering)		
Subject/Key words, S/KW :	Closed payment system, tokenization, digital wallet, blockchain, microservices architecture		
UC			
Holding data, HD :	Library, Faculty of Technical Sciences, Trg Dositeja Obradovića 6, Novi Sad		
Note, N :	None		
Abstract, AB :	This thesis explores the implementation of tokenization in closed-loop payment systems using blockchain technology, digital wallets, and tokens to enhance transaction security and data protection. It examines the advantages of closed systems, such as cost control, user loyalty, and reduced transaction costs. The proposed solution involves tokenizing the system through microservices architecture, applying hashing functions, and blockchain technology. Identified benefits include improved data security, reduced risk of breaches, and better access control. The study compares closed-loop and open-loop systems, highlighting the limitations of the proposed solution. It concludes that integrating tokenization and blockchain technology can enhance security and reduce transaction costs. Further improvements, such as recording transaction proofs on multiple blockchains and open banking, would increase flexibility and user experience.		
Accepted by the Scientific Board on, ASB :			
Defended on, DE :	November 13, 2024		
Defended Board, DB :	President:	Đorđe Pržulj, PhD, full professor	Menthor's sign
	Member:	Sonja Ristić, PhD, full professor	
	Member, Mentor:	Vladimir Mandić, PhD, associate professor	

Izjava o upotrebi generativnih AI tehnologija

Za razvoj i rad na ovoj tezi korišćeni su i priznati sledeći generativni AI alati.

Koju dozvoljenu upotrebu generativne veštačke inteligencije priznajete? ChatGPT
Koje generativne AI alate ste koristili (ime i verziju)? ChatGPT 4o mini
Za šta ste koristili alat? 1. Napredna pretraga i pojašnjavanje načina upotrebe tehnologija i tehničkih alata. 2. Stilizovanje i formatiranje teksta. 3. Za prevođenje teksta sa engleskog na srpski jezik.

Declaration of the use of generative AI technology

For the development and work on this thesis the following generative AI tools are used and acknowledged.

Which permitted use of generative AI are you acknowledging? ChatGPT
Which generative AI tool did you use (name and version)? ChatGPT 4o mini
What did you use the tool for? 1. Advanced search and clarification of the usage of technologies and technical tools. 2. Text styling and formatting. 3. For translating text from English to Serbian.

Sadržaj

1.	Uvod.....	1
1.1.	Motivacija i kontekst rešenja.....	4
1.2.	Clj i ishod rada	5
1.3.	Struktura rada	6
2.	Teorijski koncepti.....	8
2.1.	Otvoreni i zatvoreni sistemi plaćanja	8
2.2.	Digitalni novčanik i tokeni.....	10
2.3.	Blokčejn tehnologija	11
2.4.	Algoritmi i strukture podataka za tokenizaciju transakcije	12
2.4.1.	Heš funkcije.....	12
2.4.2.	Merkelovo stablo.....	15
2.5.	Mikroservisna arhitektura i modelovanje sistema upotrebom C4 modela.....	16
3.	Zahtevi rešenja	19
4.	Dizajn rešenja.....	20
4.1.	Kontekst	20
4.2.	Kontejneri.....	20
4.3.	Komponente	22
4.3.1	<i>WalletMS</i>	23
4.3.2	<i>TokenMS</i>	23
4.3.3	<i>TransactionMS</i>	25
4.3.4	<i>BlockchainMS</i>	26
5.	Implementacija rešenja.....	27
5.1	Komponente za dobavljanje, kreiranje i ažuriranje tokena određene verzije.....	27
5.2	Komponente za generisanje Merkelovog korena i validacionih putanja	31
5.3	Pametni ugovor za upis u <i>Ethereum</i> blokčejn	32
6.	Evaluacija rešenja nad <i>ZeroCash</i> sistemom.....	34
7.	Diskusija.....	36
7.1.	Ograničenja	36
7.2.	Prednosti.....	37
8.	Zaključak.....	39
	LITERATURA	41
	PRILOZI.....	45
	PRILOG A: IZVORNI KOD PROGRAM.CS KLASE API GATEWAY-A	45
	PRILOG B: IZVORNI KOD APPSETTINGS.JSON KONFIGURACIONOG FAJLA API GATEWAY-A	46
	PRILOG C: IZVORNI KOD WALLET KONTROLERA WALLET MIKROSERVISA	47

PRILOG D: IZVORNI KOD WALLET SERVISNE KLASSE WALLET MIKROSERVISA	48
PRILOG E: IZVORNI KOD TOKEN SERVISNE KLASSE WALLET MIKROSERVISA	50
PRILOG F: IZVORNI KOD REDIS SERVISNE KLASSE WALLET MIKROSERVISA	51
PRILOG G: IZVORNI KOD AZURE KEY VAULT SERVISNE KLASSE WALLET MIKROSERVISA	52
PRILOG H: IZVORNI KOD TOKEN KONTROLERA TOKEN MIKROSERVISA	52
PRILOG I: IZVORNI KOD TOKEN SERVISNE KLASSE TOKEN MIKROSERVISA	53
PRILOG J: IZVORNI KOD TRANSACTION KONTROLERA TRANSACTION MIKROSERVISA	54
PRILOG K: IZVORNI KOD TRANSACTION SERVISNE KLASSE TRANSACTION MIKROSERVISA	55
PRILOG L: IZVORNI KOD TRANSACTIONPOOL HOSTOVANE SERVISNE KLASSE TRANSACTION MIKROSERVISA	57
PRILOG M: IZVORNI KOD BLOCKCHAIN KONTROLERA BLOCKCHAIN MIKROSERVISA	58
PRILOG N: IZVORNI KOD ETHEREUM SERVISNE KLASSE BLOCKCHAIN MIKROSERVISA	59
BIOGRAFIJA	60

Spisak slika

Slika 1 Vrednost Starbucks digitalnih novčanika u odnosu na druge sisteme. Preuzeto iz [9].	3
Slika 2 Broj sajber napada u finansijskoj industriji na globalnom nivou u periodu od 2013. do 2023. godine. Preuzeto iz [14]	4
Slika 3 Struktura rada	6
Slika 4 Otvoren sistem plaćanja. Preuzeto iz [20]	8
Slika 5 Zatvoren sistem plaćanja. Preuzeto iz [20]	9
Slika 6 Blokčejn struktura podataka. Preuzeto iz [29]	11
Slika 7 Kriptografska heš funkcija. Preuzeto iz [40]	14
Slika 8 Komponente i tok procesa SHA-1. Preuzeto iz [41]	14
Slika 9 Merkelovo stablo. Preuzeto iz [43]	16
Slika 10 C4 model. Preuzeto iz [50]	17
Slika 11 Kontekst dijagram.	20
Slika 12 Arhitektura sistema zatvorenog plaćanja. Kontejner dijagram.	21
Slika 13 Komponent dijagram <i>WalletMS</i> kontejnera.	23
Slika 14 Komponent dijagram <i>TokenMS</i> kontejnera.	24
Slika 15 UML klasni dijagram <i>Factory Method</i> dizajnerskog obrazca. Preuzeto iz [51]	25
Slika 16 Komponent dijagram <i>TransactionMS</i> kontejnera.	26
Slika 17 Komponent dijagram <i>BlockchainMS</i> kontejnera.	26
Slika 18 Stanje sredstava korisnika u <i>ZeroCash</i> sistemu nakon izvršenih transakcija.	34
Slika 19 Stanje sredstava korisnika u implementiranom rešenju nakon izvršavanja transakcija.	35
Slika 20 Dokaz o upisanim transakcijama u <i>Ethereum</i> blokčejn.	35
Slika 21 Stanje u bazi transakcija implementiranog rešenja nakon izvršavanja transakcija.	35

Spisak tabela

Tabela 1 Zahtevi rešenja.....	19
Tabela 2 Tabelarni prikaz kontejnera iz C4 modela.	21
Tabela 3 Redosled izvršenih transakcija.....	34

Spisak listinga

Listing 1 <i>IToken</i> interfejs.	27
Listing 2 <i>TokenVI</i> klasa.	28
Listing 3 <i>HashFunctionVI</i> klasa.	29
Listing 4 <i>TokenDataVI</i> klasa.	30
Listing 5 <i>TokenFactory</i> klasa.	30
Listing 6 <i>MerkeRoot</i> klasa.	31
Listing 7 <i>HashFunctions</i> klasa.	32
Listing 8 Pametni ugovor za upis dokaza o izvršavanju transakcija	33

1. Uvod

U svetu savremenih finansija, gde brzina i efikasnost postaju sve važniji, zatvoreni sistemi plaćanja i digitalni novčanici predstavljaju inovativna rešenja koja transformišu način na koji se realizuju transakcije. U vremenu kada je digitalizacija postala nezaobilazni deo svakodnevnog života, razumevanje zatvorenih sistema plaćanja i digitalnih novčanika može doneti brojne koristi kako korisnicima, tako i trgovcima. Očekuje se da će vrednost sredstava u digitalnim novčanicima poput *PayPal*-a na globalnom tržištu dostići 26,41 trilion američkih dolara do 2027. godine [1]. Otvoreno bankarstvo je još jedna od savremenih tehnologija koja predstavlja unapređenje finansijske industrije omogućavajući bezbedno i lako deljenje finansijskih sredstava između banaka, nebankarskih finansijskih institucija i nezavisnih dobavljača finansijskih usluga. Vrednost transakcija nastalih putem otvorenog bankarstva širom sveta dostigla je 57 milijardi američkih dolara 2023. godine, a očekuje se da će 2027. godine dostići 330 milijardi američkih dolara. Takođe, 2023. godine broj poziva API-ja otvorenog bankarstva iznosi 102 milijarde poziva, a očekuje se da će taj broj 2027. godine biti 580 milijardi [2], [3].

Zatvoreni sistem plaćanja je finansijski okvir koji omogućavaju da se transakcije obavljaju isključivo unutar određene mreže ili ekosistema. Za razliku od otvorenih sistema, kao što su kreditne kartice, koji omogućavaju širu upotrebu kod različitih trgovaca, zatvoreni sistemi poboljšavaju kontrolu i efikasnost ograničavanjem korišćenja sredstava na transakcije unutar svog određenog okruženja. U ovim sistemima, načini plaćanja bilo da su u pitanju fizičke kartice, mobilne aplikacije digitalni novčanici ili nešto drugo, ograničeni su na određene usluge u okviru zatvorenog sistema. Ključna karakteristika zatvorenog sistema plaćanja je njegova ekskluzivnost. Transakcije se vrše korišćenjem vlasničkih kartica ili digitalnih novčanika koji su važeći samo unutar mreže. Vlasničke kartice su specijalizovane fizičke kartice koje pružaju korisnicima mogućnost da vrše transakcije unutar određenog zatvorenog sistema plaćanja, dok digitalni novčanici predstavljaju novčanike na internetu koji pružaju mogućnost korisnicima da čuvaju imovinu u digitalnom obliku. Sredstva učitana na ove načine plaćanja mogu se trošiti isključivo kod određenih trgovaca ili pružalaca usluga, čime se poboljšava kako praktičnost, tako i lojalnost unutar ekosistema. [4], [5], [6]

Digitalni novčanici su postali ključni deo svakodnevnih rutina u ovom brzo promenljivom digitalnom dobu. Oni su značajno promenili način na koji se obavljaju finansijske transakcije i upravlja sopstvenim finansijama. Na tržištu dominiraju dva glavna tipa digitalnih novčanika, a to su digitalni novčanici zatvorenog i otvorenog tipa. Razumevanje razlike između ova dva sistema je od suštinskog značaja kako bi se donela odluka o tome koji od njih najbolje odgovara potrebama sistema.

Digitalni novčanici zatvorenog tipa, poznati i kao novčanici za jedinstvenu svrhu, namenjeni su korišćenju unutar ekosistema određenog trgovca odnosno u okviru zatvorenog sistema plaćanja. Ovi novčanici ograničavaju transakcije na sistem koji ih izdaje ili na određenu grupu povezanih sistema. Glavna prednost zatvorenih novčanika je njihovo integrisanje s programima lojalnosti i nagrada trgovca. Korisnici mogu skupljati bodove ili nagrade za transakcije unutar ovog ekosistema, što podstiče ponovnu kupovinu. Takođe, sistemi koji koriste zatvorene novčanike često nude ekskluzivne popuste ili promocije, čime se dodatno poboljšava ukupno iskustvo kupaca. Otvoreni digitalni novčanici, poznati i kao novčanici za više svrha, pružaju svestranije iskustvo plaćanja. Oni omogućavaju transakcije u okviru različitih sistema što ih čini široko prihvaćenim i idealnim za svakodnevnu upotrebu. Ključna prednost otvorenih novčanika je njihova interoperabilnost. Korisnici mogu povezati različite izvore plaćanja, kao što su bankovni računi i kreditne kartice, sa svojim novčanicima. Na taj način, digitalni novčanik postaje centralno mesto za sve finansijske aktivnosti, čime se omogućava jednostavniji i efikasniji proces plaćanja, bez obzira na trgovca ili platformu. [7]

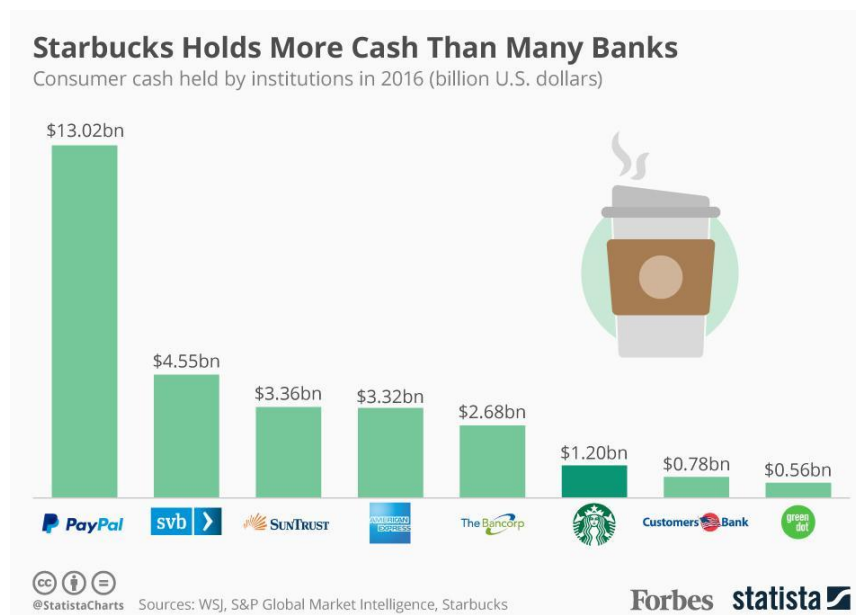
Glavni razlozi za implementaciju zatvorenih sistema plaćanja jesu brzo odobravanje transakcija, plaćanja bez kontakta (engl. *contactless*) koja olakšavaju kupovinu i koja su mnogo doprinela u procesu plaćanja tokom pandemije COVID-19 virusa, jednostavna integracija sa poslovnim alatima poput programa lojalnosti, alata za praćenje finansija i drugih. Takođe, automatsko izveštavanje omogućava sistemima da lako prate finansije i korisničke podatke, što dovodi do efikasnijeg poslovanja i većeg zadovoljstva korisnika. [4]

Neki od najpoznatijih zatvorenih sistema plaćanja na globalnom nivou su:

- **Starbucks** - uspešno je implementirao zatvoren sistem plaćanja putem svoje mobilne aplikacije gde kupci mogu da dodaju sredstva na svoje digitalne novčanike i koriste ih za plaćanje pića i proizvoda na *Starbucks* lokacijama. Ovaj sistem povećava lojalnost kupaca, smanjuje troškove transakcija i omogućava personalizovane ponude.
- **Disney MagicBand** - još jedan istaknuti primer zatvorenog sistema plaćanja implementiran kroz uređaj koji se nosi na ruci i može se koristiti za ulaz u park, prijavu u hotel, rezervacije restorana i kupovinu proizvoda unutar *Disney* ekosistema. Ovo pruža nekoliko prednosti za biznis a to su poboljšano iskustvo gostiju, prikupljanje podataka i generisanje prihoda.
- **ClassPass** - platforma za fitnes zasnovana na pretplati koja nudi pristup raznim fitnes studijima. Funkcioniše kao zatvoren sistem plaćanja tako što omogućava članovima da rezervišu časove i plaćaju putem *ClassPass* aplikacije. Ovaj sistem pruža pojednostavljen proces rezervacije, fleksibilna plaćanja i partnerstva sa studijima [4].

Jedan od poznatih primera sistema zatvorenog plaćanja na nivou Srbije jeste *Cineplexx* bonus kartica. To je sistem zatvorenog plaćanja u okviru *Cineplexx* bioskopa koji korisnicima nudi mogućnost da uplate novac na karticu nakon čega dobijaju prednosti u vidu bonus bodova kupovinom ulaznica, hrane ili pića, onlajn kupovine ulaznica, popust na kupovinu ulaznica i zamenu bonus bodova za određene nagrade. [8]

Prema podacima iz 2016. godine, *Starbucks* drži više od 1.2 milijarde američkih dolara na digitalnim novčanicima u okviru svog zatvorenog sistema plaćanja, zahvaljujući 41% kupaca u SAD-u i Kanadi koji uplaćuju unapred (engl. *deposit*) sredstava na svoje digitalne novčanike. Sa 12 miliona članova programa lojalnosti, *Starbucks* ima više novca na digitalnim novčanicima nego mnoge banke u depozitima [9]. Može se primetiti rast depozita na *Starbucks*-ovim digitalnim novčanicima i da 2022. godine ta suma iznosi 1.6 milijardi američkih dolara. [10]



Slika 1 Vrednost Starbucks digitalnih novčanika u odnosu na druge sisteme. Preuzeto iz [9].

Zatvoreni sistemi plaćanja obično imaju niže transakcione takse u poređenju sa otvorenim sistemima koji se oslanjaju na kreditne kartice i treće strane. Ove uštede mogu se preneti potrošačima kroz niže cene ili bolje ponude. Dok otvoreni sistemi nude široku prihvaćenost i veću konkurenciju, dolaze s višim taksama i manje kontrole podataka. Manje posrednika u zatvorenim sistemima često rezultira nižim troškovima, što koristi i vlasnicima i korisnicima sistema. [4], [5], [6]

Trgovci godišnje troše više od 138 milijardi dolara na takse. Na malim transakcijama dodatni troškovi mogu premašiti 12,5% vrednosti transakcije, što predstavlja značajno opterećenje za korisnika sistema i sam biznis. Zatvoreni sistemi plaćanja omogućavaju trgovcima da smanje takse, jer su sredstva već kod njih čime se izbegavaju troškovi kao što su takse procesora plaćanja kod otvorenih sistema. Plaćanja se uglavnom sastoje od varijabilnih i fiksnih taksi, pri čemu fiksne takse više utiču na manje transakcije. Na primer, 30 centi na transakciji od 5 američkih dolara predstavlja veći procenat ukupnog prihoda nego na transakciji od 100 dolara. Korišćenjem digitalnih zatvorenih novčanika, trgovci mogu smanjiti troškove i poboljšati zadovoljstvo korisnika. [10]

Jedan od najnovijih trendova u svetu finansija i digitalizacije jeste otvoreno bankarstvo (engl. open banking). Otvoreno bankarstvo predstavlja bezbedan i jednostavan način za upravljanje finansijama koji korisnicima i kompanijama omogućava lakši pristup širokom spektru finansijskih usluga. Bazira se na sigurnim sistemima i reguliše ga zakonski okvir, a omogućava pristup štednim računima, investicijama, budžetiranju i efikasnim opcijama plaćanja. Mnoge organizacije nude opciju plaćanja putem usluga otvorenog bankarstva, a korisnici ih mogu koristiti i za dopunu štednih računa ili donacije u dobrotvorne svrhe. [11] Otvoreno bankarstvo je bezbedno jer se oslanja na sigurnosne sisteme koje koriste tradicionalne banke i nove *fintech* kompanije. Korisnici nikada ne moraju da dele osetljive podatke kao što su šifre, umesto toga, koriste se metode autentifikacije poput lozinki, otiska prsta, prepoznavanja lica i SMS kodova. Servisi otvorenog bankarstva funkcionišu putem bezbednih API-ja, slično kao i standardno onlajn bankarstvo. Korisnici sami biraju koje aplikacije i veb-sajtove će povezati sa svojim računima i mogu u svakom trenutku povući saglasnost za pristup podacima. Uz enkripciju i kontrole za sprečavanje prevara, otvoreno bankarstvo omogućava sigurno upravljanje finansijama i štiti lične podatke korisnika. [12]

Narodna skupština Republike Srbije usvojila je Zakon o izmenama i dopunama Zakona o platnim uslugama u Srbiji, 31. jula 2024. godine, što donosi značajan napredak u oblasti finansijskih usluga, posebno kroz uvođenje koncepta otvorenog bankarstva. Ovo omogućava korisnicima pregled svih bankovnih računa na jednom mestu i pruža mogućnost iniciranja plaćanja putem različitih institucija, povećavajući konkurenciju i transparentnost. Banke će sada biti obavezne da uz pristanak korisnika dele minimalne podatke sa pružaocima platnih usluga, čime se otvara prostor za inovacije.

Pored toga, novi zakon uvodi pouzdanu autentifikaciju korisnika, koristeći više faktora zaštite za bezbednije elektronsko plaćanje. [13]

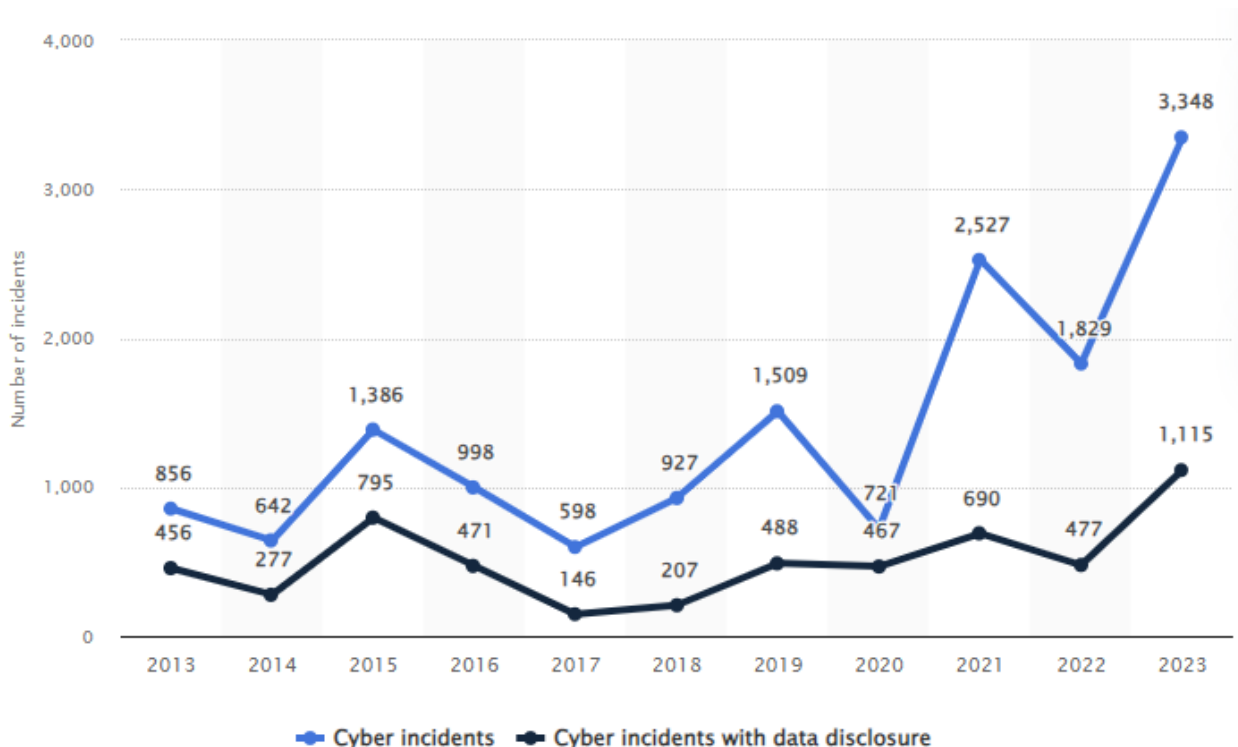
1.1. Motivacija i kontekst rešenja

Velike transakcione takse u kontekstu mana otvorenih sistema plaćanja u odnosu na zatvorene sisteme plaćanja ujedno predstavljaju i benefit u kontekstu prihoda otvorenih sistema plaćanja. Transakcione takse mogu biti značajan izvor prihoda za otvorene sisteme plaćanja, a ta sredstva se alociraju na različite oblasti, uključujući unapređenje mera bezbednosti. Investicije mogu uključivati:

- **Nadogradnja bezbednosne infrastrukture** - banke ulažu u napredne tehnologije, poput *firewall*-a, enkripcije i sistema za otkrivanje upada, kako bi se zaštitile od sajber pretnji.
- **Usaglašenost i regulative** – otvoreni sistemi za plaćanje moraju da se pridržavaju regulatornih zahteva vezanih za bezbednost, što često zahteva dodatna ulaganja u mere usaglašenosti i revizije.
- **Obuka zaposlenih** - deo profita od transakcionih taksi može se koristiti za obuku zaposlenih o bezbednosnim protokolima i svesti o prevenciji unutrašnjih prevara.
- **Istraživanje i razvoj** – kompanije vlasnici otvorenih sistema za plaćanje mogu ulagati u istraživanje radi razvoja novih bezbednosnih tehnologija ili poboljšanja postojećih sistema.

Korišćenje profita od transakcionih taksi za jačanje bezbednosti sistema pomaže otvorenim sistemima za plaćanje da zaštite podatke svojih klijenata i održe poverenje, što na kraju podržava njihovu dugoročnu održivost.

Na narednoj slici može se videti broj sajber napada u finansijskoj industriji na globalnom nivou u periodu od 2013. do 2023. godine [14].



Slika 2 Broj sajber napada u finansijskoj industriji na globalnom nivou u periodu od 2013. do 2023. godine. Preuzeto iz [14]

Može se zaključiti da broj sajber napada periodično raste uz povremena opadanja na godišnjem nivou kao i to da je 2023. godine taj broj iznosio 3348 što je najveći broj napada na godišnjem nivou za posmatrani period. Ovi napadi mogu da dovedu do povrede podataka (engl. *data breach*) što dalje može uzrokovati velikim finansijskim gubicima. Prosečna vrednost gubitka usled povrede podataka u 2023. godini u finansijskoj industriji na globalnom nivou iznosila je oko 5.9 miliona američkih dolara [15]. Jedan od najpoznatijih sajber napada sa povredom podataka u finansijskoj industriji bio je napad na kompaniju Equifax 2017. godine. Tokom ovog napada, hakeri su dobili pristup ličnim podacima oko 147 miliona ljudi, uključujući imena, adrese, datume rođenja, brojeve socijalnog osiguranja i podatke o kreditnim karticama. Ovaj incident je izazvao veliku zabrinutost zbog sigurnosti ličnih informacija i doveo do brojnih pravnih postupaka i reformi u zaštiti podataka dok je kompanija Equifax kažnjena sa kaznom od 700 miliona američkih dolara [16].

Niže transakcione takse prisutne kod zatvorenih sistema plaćanja, pored svih navedenih benefita za krajnje korisnike ujedno jesu i jedan od nedostataka tih sistema u kontekstu investicija i prihoda koji bi se mogli usmeriti na unapređenje bezbednosti sistema.

Jedno od rešenja koje ne zahteva mnogo investicija u kontekstu finansija jeste tokenizacija digitalnih novčanika zatvorenih sistema plaćanja. Ne postoji univerzalno prihvaćena definicija „tokenizacije“. *Financial Conduct Authority* (FCA) je nedavno opisala ovaj proces kao predstavljanje imovine u vidu digitalnog tokena koji je zapisan na blokčejnu ili *Distributed Ledger Technology*-u (DLT) koji omogućava pametne ugovore (tj. visoko programabilna, automatizovana i kriptografski sigurna baza podataka koja se deli između strana) [17]. U nastavku rada biće objašnjeno na koji način je dizajnirana i implementirana tokenizacija sadržaja novčanika kako bi sistem bio bezbedniji u kontekstu sprečavanja povrede podataka.

1.2. Clj i ishod rada

Cilj ovog rada, odnosno zadatka, jeste dizajnirati i implementirati sistem za tokenizaciju sistema plaćanja otvorene petlje upotrebom blokčejna, digitalnih novčanika i tokena. U okviru ovog rada, blokčejn tehnologija se koristi za čuvanje korena Merkelovog stabla generisanog transakcijama koje su nastale kreiranjem tokena određenih digitalnih valuta u okviru platnog sistema zatvorene petlje, što će biti detaljno objašnjeno u nastavku ovog rada. Pored toga jedan od ciljeva jeste da jasno prikaže koje su to prednosti i ograničenja tokenizovanih sistema plaćanja zatvorene petlje uz korišćenje blokčejn tehnologije u odnosu na tradicionalne otvorene sisteme plaćanja. Rad će se fokusirati na bezbednost transakcija upotrebom prethodno navedenih koncepata, kao i na fleksibilnost, skalabilnost i performanse sistema.

Sistem je implementiran upotrebom mikroservisne arhitekture i C# programskog jezika, odnosno .NET okvira koji pruža podršku za razvoj pojedinačnih mikroservisa. Implementirana su četiri mikroservisa *WalletMS*, *TokenMS*, *TransactionMS* i *BlockchainMS*. *WalletMS* čuva i upravlja digitalnim novčanicima i tokenima. Za čuvanje informacija kojem korisniku pripada koji digitalni novčanik koristi *Azure Key Vault* servis, dok za zaključavanje novčanika u toku izvršavanja transakcije koristi *Redis*. *TokenMS* je zadužen za kreiranje i validiranje tokena. *TransactionMS* kreira transakcije različitih tipova, dok *BlockchainMS* upisuje dokaz o izvršavanju tih transakcija u *Ethereum* blokčejn pomoću funkcionalnosti koje pruža *Nethereum*¹ biblioteka.

¹ *Nethereum*

1.3. Struktura rada

Sadržaj koji je izložen u ostatku rada je podeljen na šest sekcija kao što je prikazano na Slika 1.



Slika 3 Struktura rada

U Sekcija 2 su pojašnjeni teorijski koncepti korišćeni tokom izrade ovog rada, objašnjene su razlike između otvorenih i zatvorenih sistema plaćanja, pojmovi novčanika, tokena i blokčejn tehnologije kao i algoritmi i strukture podataka za tokenizaciju transakcije, a to su *hash* algoritmi i Merkelovo stablo. Za modelovanje sistema koristi se C4 ²model i mikroservisna arhitektura čiji su koncepti takođe pojašnjeni u okviru ove sekcije.

U Sekcija 3 nalazi se spisak zahteva rešenja koje implementirano rešenje treba da ispuni. Neki od zahteva jesu sigurnost podataka koje sadrže novčanici, tokeni i transakcije, mogućnost sistema da podrži više tipova tokena kao i različite verzije tokena. Da zabeleži izvršene transakcije u blokčeju i da obezbedi odgovarajuće performanse kreiranja transakcija.

U Sekcija 4 je detaljno prikazan dizajn sistema, odnosno kontekst, kontejneri i komponente iz već pomenutog C4 modela.

U Sekcija 5 prikazna je implementacija rešenja. Za implementaciju pojedinačnih mikroservisa koristi se C# programski jezik i .NET okvir kao glavne tehnologije za razvoj sistema, uz mnoštvo dodatnih tehnologija neophodnih za rad sistema koje će biti detaljnije objašnjene u samoj sekciji.

² C4 model

U Sekcija 6 pružena je evaluacija rešenja nad *ZeroCash* sistemom i poređenje tačnosti podataka i performansi ovog sistema i rešenja.

U Sekcija 7 nalazi se diskusija o prednostima implementacije rešenja kao i o ograničenjima koja su zapažena prilikom implementacije.

U Sekcija 8, pružen je osvrt na celokupan rad i o tome u kolikoj meri je rešenje ispunilo zadati cilj i zahteve rešenja.

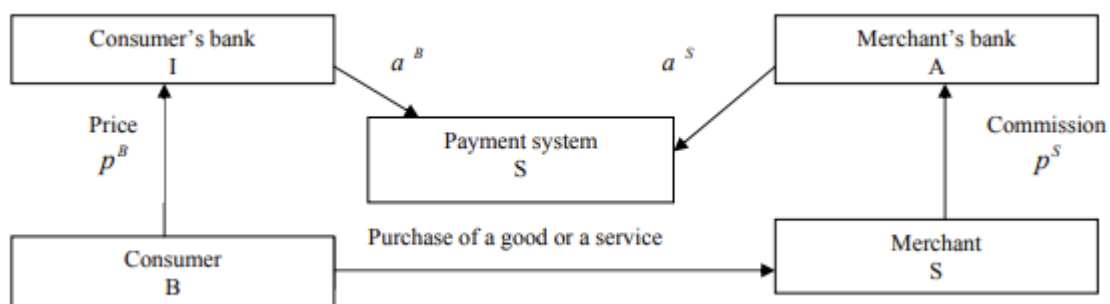
2. Teorijski koncepti

U okviru ove sekcije prvo će biti objašnjeni pojmovi otvorenih i zatvorenih sistema plaćanja kao i njihove razlike. Nakon toga biće objašnjeni pojmovi digitalnih novčanika, tokena i blokčejn tehnologije. Zatim, objašnjeni su algoritmi i strukture podataka za tokenizaciju transakcije koji se koriste u okviru ovog rešenja, to su Merkelovo stablo i heš funkcije. Na samom kraju objašnjena je mikroservisna arhitektura kao i C4 model koji je korišćen za modelovanje ovog rešenja.

2.1. Otvoreni i zatvoreni sistemi plaćanja

Sistem plaćanja definiše se kao skup mehanizama i infrastruktura koji omogućavaju sigurno, pouzdano i efikasno obavljanje transakcija [18]. U kontekstu ovog rada definišaće se dva tipa sistema plaćanja kao i njihove razlike, prednosti i mane. To su otvoreni i zatvoreni sistemi plaćanja.

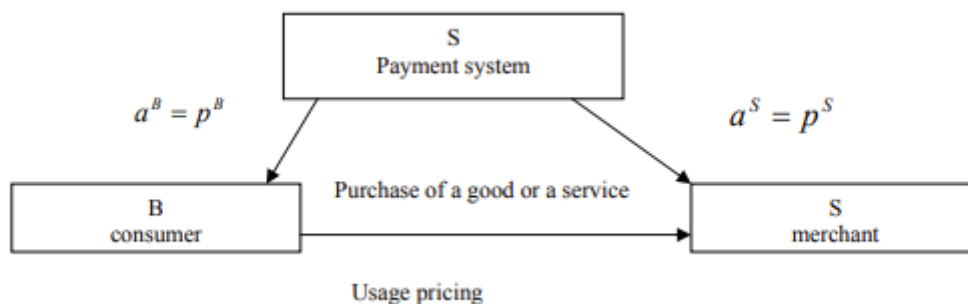
Otvoreni sistemi plaćanja predstavljaju složene finansijske strukture koje olakšavaju prenos sredstava između platitelja i primaoca koji pripadaju različitim ili istim bankama, poznate još i kao četvoro-partijski model (engl. *four-party model*) (**Slika 4**). Ovi sistemi zahtevaju angažovanje centralizovane treće strane koja je regulisana od strane finansijskih vlasti, a koja odgovara za procesuiranje i koordinaciju transakcija. Iako banke mogu vršiti direktna plaćanja, to nije uobičajeno zbog složenosti. Otvoreni sistemi plaćanja koriste različite metode naplate, uključujući fiksne i varijabilne naknade, čime se oblikuje cena usluga za krajnje korisnike, što zavisi od stepena konkurencije među bankama. Jedni od najpoznatijih primera otvorenih sistema plaćanja su *Visa* i *MasterCard*. [19], [20]



Slika 4 Otvoren sistem plaćanja. Preuzeto iz [20]

Zatvoreni sistemi plaćanja funkcionišu pod upravom jedne kompanije koja direktno sklapa ugovore sa korisnicima i trgovcima, što ih čini jednostavnijim za korišćenje. U ovom tri-partijskom

modelu (engl. *three-party model*), transakcije se autorizuju i obrađuju unutar iste platforme, što omogućava brže i jeftinije obrađivanje transakcija (**Slika 5**). Ovi sistemi obično zahtevaju da i platitelj i primalac budu na istoj platformi, a korisnici često unapred uplaćuju sredstva koja se koriste za kupovinu, čime se olakšava brza obrada transakcija. Iako su primarno namenjeni specifičnim uslugama, mogu postati široko rasprostranjeni ali dosta teže od otvorenih sistema plaćanja zbog ograničenja na obavljanje transakcija u okviru jedne kompanije. [19], [20]



Slika 5 Zatvoren sistem plaćanja. Preuzeto iz [20]

U savremenom svetu plaćanja, otvoreni i zatvoreni sistemi predstavljaju dva ključna pristupa koji se koriste za izvršenje transakcija. Svaki od ovih sistema nudi specifične karakteristike koje zadovoljavaju različite potrebe potrošača i trgovaca. Razumevanje razlika između otvorenih i zatvorenih sistema plaćanja može pomoći korisnicima da donesu informisane odluke o načinima plaćanja koji najbolje odgovaraju njihovim potrebama i preferencijama. U nastavku su istaknute glavne razlike između ova dva sistema [4], [5], [6]:

- **Mrežna infrastruktura** - zatvoreni sistemi plaćanja su ograničeni na specifične mreže ili trgovce, dok su otvoreni sistemi široko prihvaćeni među različitim mrežama.
- **Izdavanje kartica** - u zatvorenim sistemima koriste se vlasničke kartice ili aplikacije, dok otvoreni sistemi koriste standardne kartice koje izdaju banke.
- **Prihvatanje trgovaca** - zatvoreni sistemi limitiraju transakcije na određene trgovce, dok otvoreni sistemi omogućavaju transakcije kod bilo kojeg trgovca unutar mreže kartica.
- **Transakcione takse** - zatvoreni sistemi obično imaju niže transakcione troškove, što je korisno za trgovce i izdavaoce.
- **Kontrola podataka o potrošačima** - zatvoreni sistemi pružaju veću kontrolu nad podacima o potrošačima, što je korisno za marketinške i lojalnostne programe.
- **Fleksibilnost i povoljnost** - otvoreni sistemi nude veću fleksibilnost i pogodnost za potrošače.
- **Bezbednost i usklađenost** - oba sistema zahtevaju pridržavanje sigurnosnih standarda, ali otvoreni sistemi moraju ispunjavati šire bankarske i regulative kartičnih mreža.

Zatvoreni sistemi plaćanja nude prednosti kao što su smanjeni troškovi transakcija, povećana lojalnost kupaca i bolja kontrola podataka, dok su mu mane ograničena upotrebljivost i rizik od zastare ukoliko kompanija koja je vlasnik sistema propadne. S druge strane, otvoreni sistemi omogućavaju univerzalnu prihvaćenost, podstiču konkurenciju i pružaju interoperabilnost, ali se suočavaju s višim troškovima transakcija i manjom kontrolom podataka. Ove razlike ukazuju na to kako svaki sistem zadovoljava različite potrebe potrošača i trgovaca, pri čemu zatvoreni sistemi favorizuju specifične korisnike i trgovce, dok otvoreni sistemi omogućavaju širu upotrebu i fleksibilnost.

2.2. Digitalni novčanik i tokeni

Digitalni novčanik je virtuelni sistem za skladištenje koji omogućava korisnicima da čuvaju svoje identitete i digitalne akreditive za obavljanje elektronskih kupovina i izvršavanje transakcija. Sastoji se od softverskih podataka koji čuvaju sve potrebne informacije, kao što su na primer ime korisnika, podaci o karticama i adrese za kupovinu, uz zaštitu osetljivih informacija putem enkripcije. Pored toga što služe za obavljanje transakcija, digitalni novčanici se koriste i za autentifikaciju korisnika, a mogu biti i povezani sa bankovnim računima. Postoje različite kategorije digitalnih novčanika, uključujući otvorene, poluzatvorene i zatvorene novčanike, što omogućava fleksibilnost u korišćenju u kombinaciji sa drugim sistemima plaćanja. [21], [22]

Tri kategorije digitalnih novčanika [23] :

- **Otvoreni digitalni novčanik** - vrsta novčanika koja omogućava korisnicima da izvršavaju transakcije u sistemima gde su oni prihvaćeni. Ova vrsta novčanika može se koristiti samo u saradnji sa bankama, a korisnici mogu slati novac na bilo koji bankovni račun. Samo banke imaju pravo da izdaju otvorene digitalne novčanike, čime se osigurava sigurnost i pouzdanost ovih transakcija.
- **Poluzatvoreni digitalni novčanici** - omogućavaju korisnicima da obavljaju transakcije sa svim servisima koji su u partnerstvu sa kompanijom koja je vlasnik sistema za digitalni novčanik kao i prenos sredstava između korisnika unutar mreže, ali ne omogućavaju podizanje gotovine sa digitalnog novčanika.
- **Zatvoreni digitalni novčanici** - novčanici koji se izdaju potrošačima za ekskluzivnu upotrebu kod određenih kompanija. Ovi novčanici se mogu koristiti samo za transakcije unutar kompanije koja ih izdaje, odnosno samo sa članovima koji pripadaju tom sistemu. Takođe, zatvoreni novčanici se koriste često za poklon kartice, čime se dodatno olakšava kupovina unutar specifičnih platformi.

U domenu blokčejna, tokeni se koriste za predstavljanje digitalnih sredstava, iako koncept tokena nije isključivo vezan za blokčejn. Istorijski, tokeni su služili za obezbeđivanje digitalnih transakcija, poput bankarskih. Tehnički, tokeni su deo digitalnih informacija koji detaljno objašnjavaju šta predstavljaju, dok sa pravnog aspekta mogu označavati digitalna sredstva koja imaju dogovorenu vrednost, kao što su kriptovalute kao najpoznatiji primer. Tokeni takođe mogu predstavljati određena prava, poput prava pristupa, i mogu se koristiti za tokenizaciju sredstava iz stvarnog (npr. nekretnine, kolekcionarski predmeti) i virtuelnog sveta (npr. kriptovalute, loto karte). Digitalni objekti se mogu klasifikovati kao opipljivi i neopipljivi, pri čemu opipljivi objekti imaju fizičko postojanje, dok neopipljivi predstavljaju usluge ili virtuelne predmete. U kontekstu zamenljivosti (engl. *fungibility*), digitalni objekti se dele na zamenljive (engl. *fungible*), poluzamenljive (engl. *semi-fungible*) i nezamenljive (engl. *non fungible*). U literaturi se navode različite vrste tokena, uključujući platne tokene, utilitarne tokene i alternativne tokene. Platni tokeni su, na primer, kriptovalute, dok utilitarni tokeni omogućavaju pristup određenim uslugama. Zamenljivost se definiše kao sposobnost sredstva da se zameni sličnim sredstvima. Zamenljivi tokeni imaju istu vrednost, dok nezamenljivi tokeni imaju jedinstvene karakteristike. [24]

Zamenljivost tokena se odnosi na njegovu sposobnost da bude zamenljiv ili identičan drugim zamenljivim tokenima iste kategorije. To znači da se zamenljivi tokeni (engl. *fungible tokens*) mogu lako zameniti drugim sredstvima iste ili ekvivalentne vrednosti, pri čemu se mogu deliti ili razmenjivati. Oni su identični jedni drugima i mogu se razdvojiti na manje jedinice, što ne utiče na njihovu vrednost. Na primer, platni token je uvek zamenljiv, deljiv i nije jedinstven. [24]

Nezamenljivi token (engl. *Non fungible token*) je kriptografski jedinstven token koji se koristi za praćenje vlasništva nad pojedinačnim sredstvima. Za razliku od zamenljivih tokena, nezamenljivi tokeni se ne mogu deliti i svaki sadrži jedinstvene informacije koje ga razlikuju od drugih. Standard ERC-721 omogućava kreiranje ovih tokena na *Ethereum* blokčejnu, naglašavajući razlike između

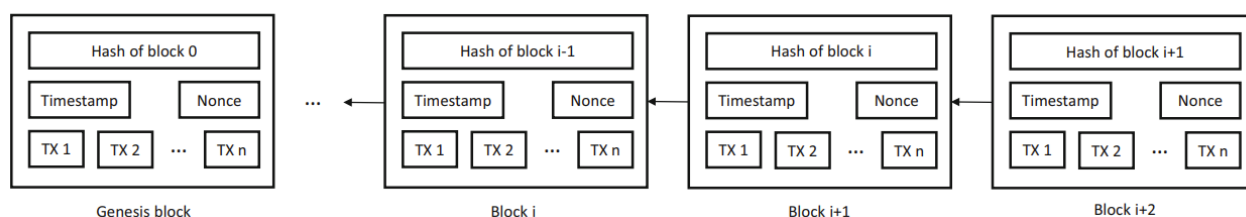
zamenljivih i nezamenljivih tokena. Dok se zamenljivi tokeni mogu lako razmenjivati bez promene vrednosti, nezamenljivi tokeni su jedinstveni i ne mogu se deliti ili spajati. Nezamenljivi token simbolizuje vlasništvo nad digitalnim ili fizičkim sredstvima, pružajući verifikovanu nepromenljivost i autentičnost zahvaljujući blokčejn tehnologiji. Ovi tokeni omogućavaju praćenje vlasništva i igraju ključnu ulogu u skaliranju *Ethereum*-ovih kapaciteta za obradu transakcija. [25], [26], [27]

Poluzamenljivi tokeni (engl. *Semi-fungible tokens*) su nova klasa tokena koja kombinuje karakteristike zamenljivih i nezamenljivih tokena, omogućavajući fleksibilnije prikazivanje složenih sredstava ili procesa. [24]

2.3. Blokčejn tehnologija

Distributed Ledger Technology (DLT) je tehnologija koja se često meša sa blokčejnom, omogućava transparentno i decentralizovano upravljanje podacima bez centralne vlasti. Bitno je napomenuti da svaki blokčejn jeste DLT, ali ne i obrnuto, nije svaki DLT blokčejn. Dok su DLT i blokčejn prvobitno zamišljeni za digitalne valute, treba napomenuti i to da su istraživači otkrili brojne druge primene ove tehnologije [28].

Blokčejn je struktura podataka koja se sastoji od lanca blokova, pri čemu svaki blok sadrži više transakcija, vremensku oznaku, heš vrednost prethodnog bloka i *nonce* za verifikaciju tog heša (**Slika 6**) Ovaj sistem osigurava integritet lanca, a jedinstvene heš vrednosti sprečavaju prevaru jer svaka promena u bloku odmah menja njegovu heš vrednost. Decentralizovani sistem, poput blokčejna, donosi brojne prednosti, kao što su otpornost na kvarove određenih čvorova, povećano poverenje među učesnicima i bolja sigurnost podataka, jer nije potrebno oslanjati se na posrednike. [29], [30], [31]



Slika 6 Blokčejn struktura podataka. Preuzeto iz [29]

Blokčejn tehnologija osigurava da podaci zapisani u blokčejn budu gotovo nepromenljivi, što je jedna od ključnih karakteristika ove tehnologije. To se postiže korišćenjem nekoliko tehnoloških mehanizama. Kriptografsko heširanje omogućava svakom bloku u blokčejnu da sadrži jedinstveni heš prethodnog bloka, pa bilo kakva promena u podacima menja heš i figurativno lomi lanac. Konsenzus mehanizmi poput dokaza o radu (engl. *proof of work*) i dokaza o ulogu (engl. *proof of stake*) zahtevaju saglasnost većine učesnika u mreži za dodavanje novog bloka, što sprečava maliciozne aktere da menjaju podatke jer bi morali da kontrolišu većinu mreže, što je skupo i složeno. Decentralizacija obezbeđuje da se baza podataka čuva na više računara širom mreže, pa čak i ako neko izmeni podatke na jednom, neslaganje će biti otkriveno i mreža će odbaciti promenu. Pametni ugovori dodatno osiguravaju transakcije automatskim izvršavanjem dogovora bez posrednika, a nakon što su postavljeni, teško ih je menjati. Merkelovo stablo omogućava brzo proveravanje integriteta velikih skupova transakcija, jer svaka promena menja koren stabla, što odmah alarmira mrežu da je došlo do manipulacije. Ove tehnologije zajedno čine blokčejn bezbednim i nepromenljivim sistemom, gde bi promena zapisa zahtevala izmenu svakog narednog bloka, što je računarski i finansijski gotovo nemoguće, naročito u velikim mrežama.

Trenutni blokčejn sistemi se grubo dele na tri tipa, javni, privatni i *consortium* blokčejn. U javnom blokčejnu, svi podaci su vidljivi javnosti, a svako može učestvovati u procesu konsenzusa. U *consortium* blokčejnu, samo odabrani čvorovi učestvuju u konsenzusu i iz tog razloga smatra se

delimično centralizovanim. Privatni blokčejn, s druge strane, kontroliše jedna organizacija i smatra se centralizovanim. [30]

Sa brzim razvojem industrije e-trgovine i onlajn finansija, milijarde mrežnih transakcija se izvršavaju na internetu u svakom trenutku. Kako obezbediti sigurnost ovih transakcija i kako postići sigurno čuvanje podataka, razmenu i deljenje masovnih podataka o transakcijama postalo je jedno od glavnih pitanja u svetu. Tehnologija blokčejna predstavlja praktično rešenje za ove probleme zbog čega dolazi i do njenog ubrzanog razvoja. Postoji više od 4.900 digitalnih kriptovaluta i više od 1.500 tokena zasnovanih na tehnologiji blokčejna. Tržišna kapitalizacija blokčejna 2021. godine dostigla je 266 milijardi američkih dolara. Gartnerova prognoza ukazivala je na to da će transakcije zasnovane na blokčejnu premašiti 10 milijardi američkih dolara do 2022. godine, a da će se do 2024. godine duplirati odnosno iznositi 20 milijardi američkih dolara. Takođe, 10% globalnog BDP-a biće pohranjeno putem tehnologije blokčejna do 2025. godine. [32]

Pregledom statistike na zvaničnom sajtu blokčejn tehnologije, može se zaključiti da vrednost transakcija zasnovanih na blokčejnu 1. marta 2024. godine iznosi oko 19.6 milijardi američkih dolara, što odgovara prethodnoj predikciji [33]. Prema tome, uz procenu da će globalno blokčejn tržište rasti po složenoj godišnjoj stopi od 68.4 % može se očekivati da će vrednost transakcija zasnovanih na blokčejn tehnologiji iznositi oko 67.4 milijarde američkih dolara [34]

Prethodno istraživanja o proceni vrednosti transakcija zasnovanih na blokčejn tehnologiji može se usko povezati i sa globalnim investicijama u kripto i blokčejn. Primećuje se da su investicije dramatično porasle sa 5,4 milijarde američkih dolara u 2020. godini na rekordnih 30 milijardi dolara u 2021. godini, dok je broj dogovora (engl. *deals*) porastao sa 627 na 1.332 u istom periodu. Zabeleženi su brojni veliki dogovori, uključujući povećanje od jedne milijarde američkih dolara od strane FTX kompanije sa Bahama, povećanje od 767 miliona američkih dolara od strane američke kompanije NYDIG i povećanje od 750 miliona američkih dolara od strane kompanije *Celsius Network*. Rastuća aktivnost investicija i dogovora odražava sve veće prepoznavanje potencijalne uloge kriptovaluta i njegovih osnovnih tehnologija u savremenim finansijskim sistemima [35]

Jedan od najpoznatijih javnih blokčejnova jeste *Ethereum*. Ovaj blokčejn koristi se prilikom implementacije ovog rešenja, a način na koji je upotrebljen objašnjen je detaljnije u nastavku ovog rada. *Ethereum* je blokčejn koji ima ugrađeni *Turingcomplete* programski jezik. Pruža apstraktni sloj koji omogućava svima da kreiraju svoja pravila za vlasništvo, formate transakcija i funkcije prelaza stanja. Ovo se postiže korišćenjem pametnih ugovora (engl. *smart contracts*), skupa kriptografskih pravila koja se izvršavaju samo ako su ispunjeni određeni uslovi. [36], [37]

Nethereum je biblioteka za .NET aplikacije koja pruža mnoštvo funkcionalnosti za interakciju sa pametnim ugovorima i *Ethereum* klijentima. Neke od funkcionalnosti koje su značajne u kontekstu ovog rada jesu to da ova biblioteka omogućava dobavljanje pametnog ugovora i pozivanje njihovih metoda upotrebom .NET-a, kao i kreiranje blokčejn transakcije i njenog slanja na *Ethereum* blokčejn.

2.4. Algoritmi i strukture podataka za tokenizaciju transakcije

U ovoj sekciji objašnjeni su pojmovi heš algoritama i strukture podataka Merkelovo stablo koji su neophodni za razumvanje rada.

2.4.1. Heš funkcije

Heš funkcija je matematička metoda koja mapira skup A u skup B , pri čemu se često pretpostavlja da je broj elemenata u skupu A veći od broja elemenata u skupu B . Ove funkcije se koriste za generisanje heš kodova koji služe kao jedinstveni identifikatori za podatke. Pretpostavimo da su x i y elementi skupa A i neka postoji neka heš funkcija f , ako se heširanjem x i y funkcijom f kao rezultati dobiju isti heš kodovi, odnosno $f(x) = f(y)$ smatra se da je došlo do kolizije. [38] Bitna

karakteristika dobre heš funkcija jeste da za dva jako slična ulaza x i y , pretpostavimo da su ova dva ulaza nizovi karaktera i da se razliku u samo jednom karakteru, funkcija treba kao izlaz da kreira dve heš vrednosti drastično različite.

Postoje tri različite grupe metoda za definisanje heš funkcije [39]:

- Metod deljenja
- Metod množenja
- Metod univerzalnog hešovanja

Metod deljenja je najjednostavniji oblik heš funkcije kojim se uzastopni ključevi mapiraju na uzastopne slotove u tabeli. Matematički ova funkcija se može predstaviti sledećom formulom:

$$h(k) = k \bmod m$$

gde je k ključ koji se hešuje, a m broj slotova u tabeli. Ovako definisana funkcija može biti dobra ako je prost broj i nije blizu stepena dva ili deset. Upravo zbog ovoga često se koristi i sledeća formula:

$$h(k) = (k \bmod p) \bmod m$$

gde je p prost broj i $p > m$.

Metod množenja je oblik heš funkcije baziran na multiplikaciji niza bitova. Podrazumeva korišćenje mašine sa dužinom reči od w bita. Neka je a proizvoljna vrednost i postmatra se njen binarni zapis dužine w . Onda je ovu heš funkciju matematički moguće prikazati formulom:

$$h(k) = [(a \cdot k) \bmod 2^w] \gg (w - r)$$

gde $\gg$ predstavlja pomeranje bita u desno i broj slotova jednak je 2^r .

Metod univerzalnog hešovanja je metod koji zadovoljava upotpunosti karakteristike jednostavnog i uniformnog hešovanja. Ako je U univerzum ključeva ključeva, heš funkciju matematički možemo predstaviti sa formulom:

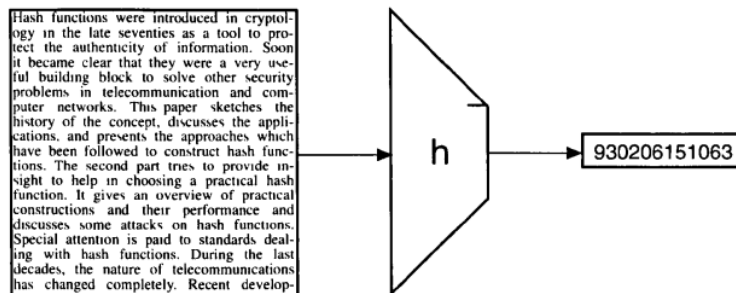
$$h(k) = [(a \cdot k + b) \bmod p] \bmod m$$

gde p predstavlja veliki prost broj, idealno $p \geq |U|$, slučajne konstante $a \in \{1, 2, \dots, p-1\}$ i $b \in \{0, 1, \dots, p-1\}$. Kod ovako definisane funkcije verovatnoća kolizije dva ključa može se predstaviti formulom:

$$Pr_{a,b}\{h(k_1) = h(k_2)\} = \frac{1}{m}$$

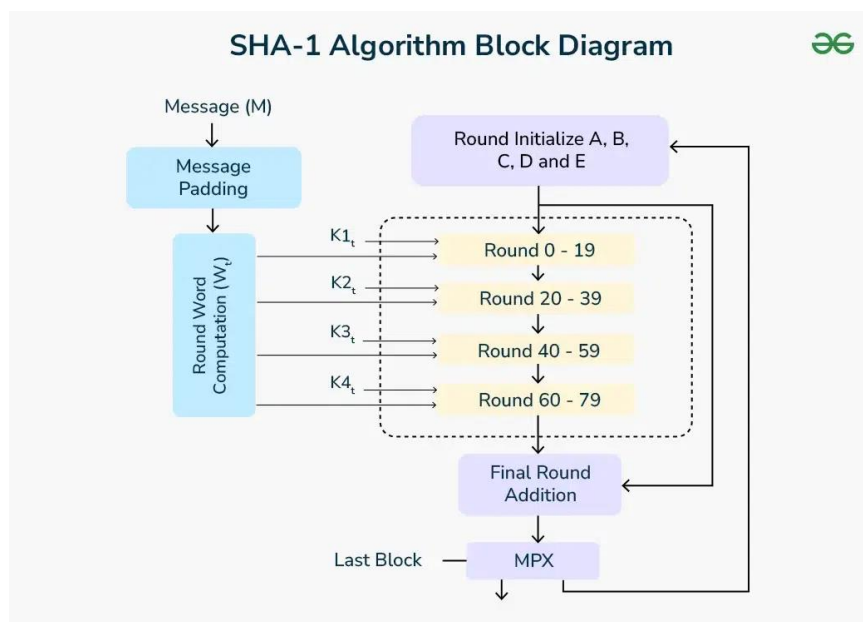
gde važi da je $k_1 \neq k_2$.

Kriptografska heš funkcija predstavlja matematičku funkciju koja uzima ulazne podatke (poruku) i generiše kratak niz fiksne dužine, nazvan heš kod, koji predstavlja tu poruku (**Slika 7**). Ova funkcija je dizajnirana tako da bude jednosmerna, odnosno u idealnom slučaju matematički gledano da ne postoji inverzna funkcija, što znači da je teško rekonstruisati ulazne podatke iz heš koda. Takođe, kriptografska heš funkcija treba da bude otporna na kolizije, što znači da je teško pronaći dva različita ulaza koja daju isti heš kod. [40] Još jedna bitna karakteristika kriptografske heš funkcije jeste to da se za dva ista ulaza dobijaju različite heš vrednosti čime se sprečava prepoznavanje duplikata. Ovo se postiže *salting*-om, odnosno dodavanjem nasumičnog podatka ulazu prilikom kreiranja heš vrednosti. Poslednja karakteristika jeste nesavitljivost kriptografske heš funkcije, odnosno nemogućnost da se promenom u izlazu dođe do informacija o ulazu, što obezbeđuje očuvanje integriteta podataka ulaza.



Slika 7 Kriptografska heš funkcija. Preuzeto iz [40]

SHA-1 (*Secure Hash Algorithm 1*) je kriptografski algoritam koji uzima ulazne podatke i generiše heš vrednost dužine 160 bita (20 bajtova), poznatu kao kratki prikaz poruke (engl. *message digest*). Ova vrednost se obično prikazuje kao heksadecimalni broj od 40 cifara. Od 2005. godine, SHA-1 se smatra nesigurnim, a veliki tehnološki divovi, uključujući *Microsoft*, *Google*, *Apple* i *Mozillu*, prestali su da prihvataju SHA-1 SSL sertifikate do 2017. godine. Komponente i tok procesa algoritma prikazani su na slici ispod (**Slika 8**). [41]



Slika 8 Komponente i tok procesa SHA-1. Preuzeto iz [41]

Komponente i tok procesa SHA-1 sastoje se iz osam ključnih delova [41]:

1. **Poruka (M)** - Originalna ulazna poruka koja treba da se hešira.
2. **Dodavanje poruke** - početni korak u kojem se poruka dodaje tako da njena dužina bude kongruentna 448 mod 512. Ovaj korak priprema poruku za obradu u blokovima od 512 bita.
3. **Računanje reči za runde (W_t i tW_t)** - nakon dodavanja, poruka se deli na blokove od 512 bita, a svaki blok se dalje deli na 16 reči od 32 bita. Ove reči se zatim proširuju na 80 reči od 32 bita koje se koriste u narednim rundama.
4. **Inicijalizacija rundi (A, B, C, D i E)** - inicijalizacija pet radnih varijabli (A, B, C, D i E) sa specifičnim konstantnim vrednostima. Ove varijable se koriste za iterativno izračunavanje heš vrednosti.
5. **Konstante rundi (K_t i tK_t)** - SHA-1 koristi četiri konstantne vrednosti (K1, K2, K3, K4), koje se primenjuju u specifičnim opsezima rundi:

6. **Runde (0-79)** - glavna petlja izračunavanja SHA-1 podeljena je u četiri faze (svaka odgovara jednoj od konstanti K1 do K4). U svakoj rundi se kombinuju logičke funkcije i operacije nad radnim varijablama (A, B, C, D i E) koristeći reči generisane u prethodnom koraku.
7. **Zbir finalne runde** - nakon svih 80 rundi, rezultatne vrednosti A, B, C, D i E se sabiraju sa originalnim heš vrednostima kako bi se dobio konačni heš.
8. **MPX (*Multiplexing*)**: Kombinuje rezultate iz zbira finalne runde da bi formirao konačni kratki zapis poruke.

U implementaciji ovog rada koristi se SHA-256 koji se razlikuje od SHA-1 u tome što generiše heš vrednost dužine 256 bita (32 bajta), ima 64 runde umesto 80 i heš vrednost se prikazuje kao heksadecimalni broj od 64 cifre. Povećanje dužine heš vrednosti na 256 bita omogućava veću otpornost na kolizije što ga čini sigurnijim i pouzdanijim za korišćenje od SHA-1.

2.4.2. Merkelovo stablo

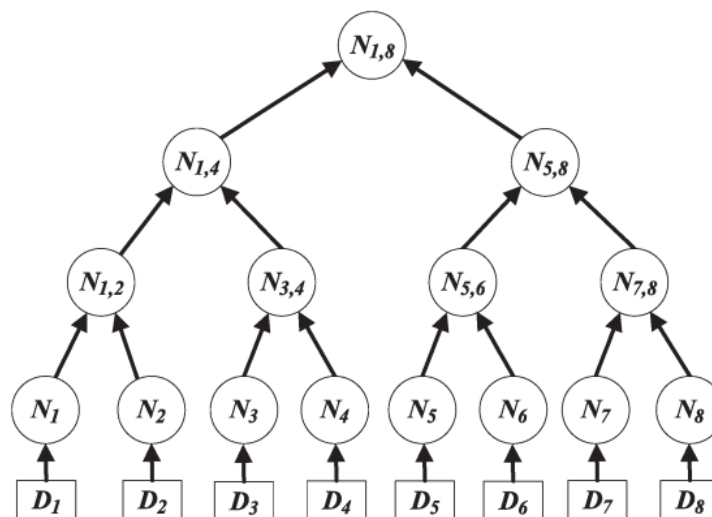
Merkelovo stablo je kriptografska struktura podataka koju je 1979. godine uveo Ralph Merkle kao deo svojih istraživanja o javnim ključevima i digitalnim potpisima. Ova inovativna ideja omogućila je efikasno verifikovanje podataka, gde su heš funkcije korišćene za povezivanje listova sa korenom stabla, čime se značajno smanjila količina potrebnih podataka za verifikaciju listova. Njihova otpornost na manipulacije i sposobnost da efikasno provere integritet podataka čine ih izuzetno važnim u savremenoj kriptografiji. [42], [43]

Merkelovo stablo je kompletno binarno stablo, gde svaki čvor sadrži heš vrednost koja se računa na osnovu heš vrednosti svojih podređenih čvorova (engl. *childs*) izuzev listova (engl. *leaves*). Matematički može se predstaviti sledećom formulom:

$$\varphi(n_{parent}) = \text{hash}(\varphi(n_{left}) || \varphi(n_{right}))$$

gde je φ zadatak koji mapira set čvorova na set stringova dužine k : $n \rightarrow \varphi(n) \in \{0,1\}^k$. Ovaj zadatak mora biti uvek zadovoljen [42]. Za heš funkciju treba odabrati neku kriptografsku heš funkciju kao što je na primer SHA-1 koja je objašnjena u prethodnoj sekciji.

Na primer, vrednost čvora $N_{1,2}$ računa se kao heš kombinacija vrednosti N_1 i N_2 (**Slika 9**). Da bi se verifikovao neki list, potrebno je samo izračunati putanju do korena, koristeći informacije o autentifikaciji koje uključuju heš vrednosti braće (engl. *siblings*) čvorova na tom putu [43]. Autentifikaciona putanja α za list N_1 predstavlja niz heš vrednosti sledećih čvorova: $\alpha : \{N_2, N_{3,4}, N_{5,8}\}$. Koristeći ovu putanju moguće je preračunati vrednost korena Merkelovog stabla sa konstantnom vremenskom kompleksnošću i uporediti sa stvarnom vrednošću korena, ukoliko su heš vrednosti jednake smatra se da je list N_1 validan.



Slika 9 Merkelovo stablo. Preuzeto iz [43]

2.5. Mikroservisna arhitektura i modelovanje sistema upotrebom C4 modela

Mikroservisna arhitektura postala je dominantan izbor u industriji softverskih usluga. Ovaj stil arhitekture naglašava deljenje sistema na male i lagane servise koji su specifično dizajnirani za izvršavanje kohezivnih poslovnih funkcija, predstavljajući evoluciju tradicionalne arhitekture zasnovane na uslugama. Mikroservis se definiše kao minimalan nezavisan proces koji komunicira putem poruka, a mikroservisna arhitektura kao distribuirana aplikacija sastavljena od tih mikroservisa [44].

Za implementaciju mikroservisnih arhitektura trenutno se koriste tehnologije kontejnera, pri čemu se svaki komponenta pokreće u nezavisnom kontejneru. Kontejneri pružaju lagano okruženje za izvršavanje, pakujući aplikaciju i sve njene zavisnosti na potpuno izolovan način, što savršeno odgovara potrebama mikroservisa [45], [46]. Takođe tehnologije kao što su *cloud computing* i virtualizacija imaju važnu ulogu u mikroservisnoj arhitekturi. *Cloud computing* je tehnologija koja koristi internet i centralne udaljene servere za održavanje podataka i aplikacija, omogućavajući korisnicima da koriste aplikacije bez instalacije i pristupe ličnim fajlovima sa bilo kog računara sa internet konekcijom, čime se značajno poboljšava efikasnost centralizacijom resursa kao što su skladište, memorija, procesiranje i protok podataka. Ključna komponenta ove tehnologije je virtualizacija, koja predstavlja apstrakciju resursa kao što su procesor, skladište, mreža i memorija od aplikacija i krajnjih korisnika, omogućavajući *multitenancy* poslovne modele u *cloud* okruženju. [47] *Multitenancy* predstavlja način rada softvera u kojem više nezavisnih instanci jedne ili više aplikacija funkcioniše u zajedničkom okruženju.

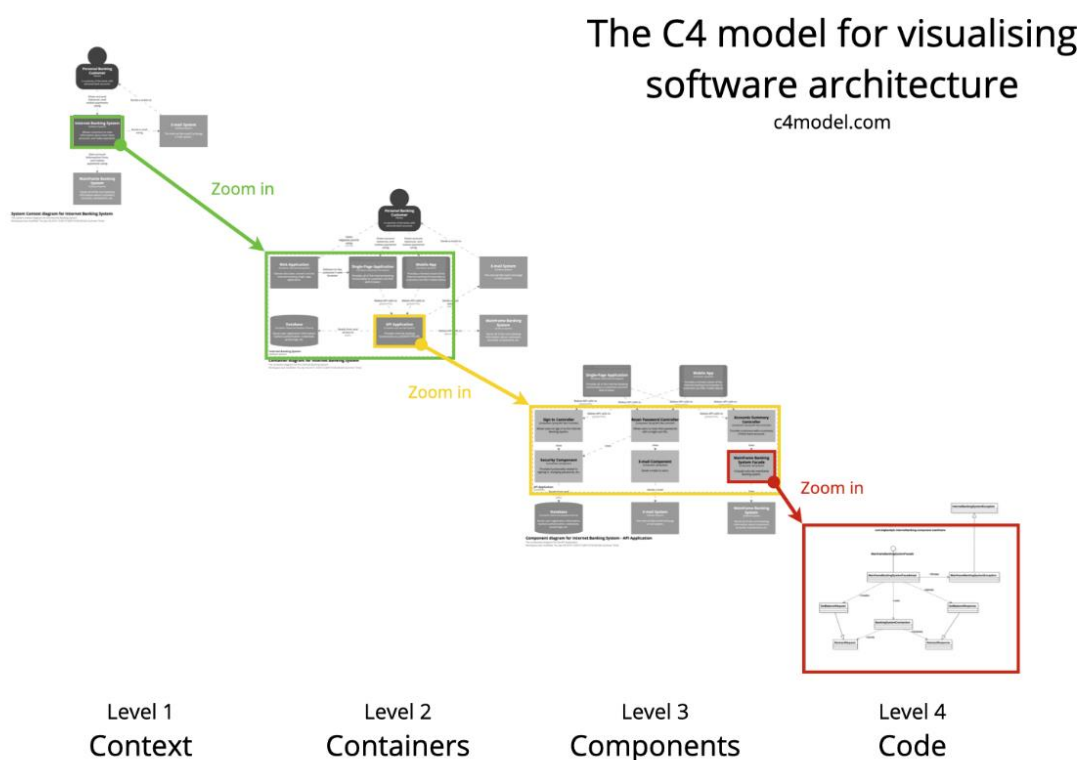
Mikroservisna arhitektura nudi brojne prednosti. Ključna prednost je održivost; razbijanjem sistema na nezavisne servise, timovi mogu testirati i menjati delove sistema bez ometanja drugih, čime se pojednostavljuje distribuirani razvoj. Takođe, mala veličina mikroservisa povećava razumljivost koda. Skalabilnost je još jedna važna prednost; mikroservisi se mogu rasporediti na različitim mašinama i lako se mogu skalirati po potrebi, bez potrebe za skupim serverima. Zamenljivost je takođe jednostavna, jer su mikroservisi manji, što olakšava njihovu zamenu ili brisanje. Tolerancija na greške je još jedan ključni benefit; kvar jednog mikroservisa obično ne utiče na ceo sistem, a oštećeni mikroservisi se brzo mogu ponovo pokrenuti. Pouzdanost se takođe poboljšava, jer se složeni sistemi grade od jednostavnih komponenti sa jasnim interfejsima. [48]

U kontekstu čuvanja stanja, mikroservise je moguće podeliti na one koji čuvaju stanje (engl. *stateful*) tokom rada servisa i one koji ne čuvaju (engl. *stateless*). *Stateless* mikroservis ne zavisi od specifičnog stanja koje mora da zadrži što omogućava njegovo lako skaliranje, odnosno podizanje više instanci, jer se može pokretati kao zamenljiva instanca. Međutim, to nije slučaj sa *stateful* mikroservisima. Instance *stateful* mikroservisa imaju različita stanja, zbog čega nisu međusobno zamenljive. Implementacija replikovanog skupa *stateful* mikroservisa zahteva koordinaciju između različitih replika kako bi ostale sinhronizovane što otežava njegovo skaliranje, odnosno podizanje više instanci, za rad boljih performansi. [49] *Stateless* mikroservisi su obično skalabilniji, jer svaki zahtev može da obradi bilo koja dostupna instanca mikroservisa, dok *stateful* mikroservisi zahtevaju složenije mehanizme za balansiranje opterećenja i upravljanje sesijama. *Stateless* mikroservisi su otporniji na kvarove, jer gubitak jedne instance ne utiče na korisničke sesije, dok u *stateful* mikroservisima gubitak servera može dovesti do gubitka podataka o sesiji. Takođe, *stateless* mikroservisi često koriste manje resursa i jednostavniji su za razvoj i održavanje, jer nema potrebe za upravljanjem stanjem kroz više zahteva, dok *stateful* mikroservisi zahtevaju pažljivo upravljanje podacima sesije i stanjem.

C4 model je pristup modelovanju softverske arhitekture koji akcentuje četiri perspektive CCCC koji se sastoji od:

- Skupa hijerarhijskih apstrakcija koje obuhvataju softverske sisteme, kontejnere, komponente i kod.
- Skupa hijerarhijskih dijagrama koji uključuju kontekst sistema, kontejnere, komponente i kod.
- Nezavisnosti od notacije, što znači da se može koristiti sa različitim notacijama.
- Nezavisnosti od alata, što omogućava korišćenje raznih alata za izradu dijagrama.

C4 model olakšava razumevanje i komunikaciju o strukturi softverskog sistema na različitim nivoima detalja. [50]



Slika 10 C4 model. Preuzeto iz [50]

Kontekst, odnosno softverski sistem, predstavlja najviši nivo apstrakcije i opisuje nešto što donosi vrednost korisnicima, bilo ljudima ili ne. To uključuje softverski sistem koji modelirate, kao i druge sisteme na kojima vaš sistem zavisi ili obrnuto. Nažalost, termin "softverski sistem" je najteže definisati u C4 modelu, a situaciju dodatno komplikuje to što svaka organizacija ima svoju terminologiju, koristeći izraze kao što su aplikacija, proizvod, usluga, itd. Može se reći da je softverski sistem nešto što jedna razvojna ekipa gradi, poseduje, ima odgovornost za njega i može da vidi njegove unutrašnje implementacione detalje.

U C4 modelu, kontejner predstavlja aplikaciju ili skladište podataka i neophodan je za funkcionisanje celog softverskog sistema. Kontejner može biti različitih tipova, uključujući:

- **Serverski orijentisane veb aplikacije** – na primer *Java EE* aplikacije na *Apache Tomcat*, ASP.NET MVC na *Microsoft IIS*, *Ruby on Rails* na *WEBrick* ili *Node.js* aplikacije.
- **Klijentski orijentisane veb aplikacije** – na primer *JavaScript* aplikacije koje se pokreću u veb pregledaču koristeći *Angular*, *React*, *Vue.js* itd.
- **Desktop aplikacije** – na primer *Windows* aplikacije napisane u WPF, OS X aplikacije u *Objective-C* ili multi platformskih aplikacija u *JavaFX*.
- **Mobilne aplikacije** - aplikacije za *Apple iOS*, *Android* ili *Microsoft Windows Phone*.
- **Serverski orijentisane konzolne aplikacije** - samostalne aplikacije ili *batch* procesi.
- **Serverless funkcije** – na primer *Amazon Lambda* ili *Azure Function*.
- **Baze podataka** – na primer *MySQL*, *Microsoft SQL Server*, *Oracle Database*, *MongoDB* itd.
- **Blob ili content skladišta** – na primer *Amazon S3* ili *Azure Blob Storage*.
- **Fajl sistemi** - lokalni fajl sistem ili deo većeg umreženog fajl sistema.
- **Shell skripte** - skripte napisane u *Bash*-u itd.

Kontejner predstavlja granicu izvršavanja oko koda koji se pokreće ili podataka koji se skladište. Ime "kontejner" je odabrano da ne implicira fizičku prirodu izvršavanja. Na primer, jedan *Java EE* server kao što je *Apache Tomcat* može pokretati više veb aplikacija unutar jedne *Java Virtual Machine*, pri čemu su te aplikacije izolovane jedna od druge. Tokom razvoja, može biti više veb aplikacija na jednom serveru, dok se svaka aplikacija može *deploy*-ovati na poseban server u produkciji. U tom slučaju, svaka veb-aplikacija se smatra C4 kontejnerom, dok je proces *deploy*-ovanja odvojena briga.

Reč komponenta ima različita značenja u industriji razvoja softvera, ali u C4 modelu, komponenta predstavlja grupisanje povezanih funkcionalnosti koje su enkapsulirane iza jasno definisanog interfejsa. Ako koristite jezike kao što su *Java* ili *C#*, komponentu možete zamisliti kao kolekciju implementacionih klasa koje su povezane s interfejsom. U C4 modelu, komponente nisu odvojene jedinice koje se mogu *deploy*-ovati. Umesto toga, *deploy*-ovana jedinica je kontejner, što znači da sve komponente unutar kontejnera izvršavaju u istom procesnom prostoru. Aspekti kao što su pakovanje komponenti (na primer, jedna komponenta naspram više komponenti po JAR datoteci, DLL-u, deljenoj biblioteci itd.) su odvojena briga.

Konačno na najnižem nivou apstrakcije dolazimo do kod dela. Komponente se sastoje od jednog ili više kodnih elemenata koji su konstruisani od osnovnih građevinskih blokova programskog jezika koji koristite, kao što su klase, interfejsi, enumeracije, funkcije, objekti itd.

3. **Zahtevi rešenja**

U okviru ove sekcije tabelarno su prikazani osnovni zahtevi koje implementirano rešenje treba da zadovolji kako bi se ispunio prethodno objašnjen cilj rešenja u Sekcija 1. Način na koji su delovi sistema dizajnirani i implementirani kako bi se ovi zahtevi ispunili, a samim tim i cilj rešenja, biće objašnjeni u Sekcija 4 i Sekcija 5. U narednoj tabeli su prikazani oznaka i opis svakog od zahteva rešenja.

Tabela 1 Zahtevi rešenja.

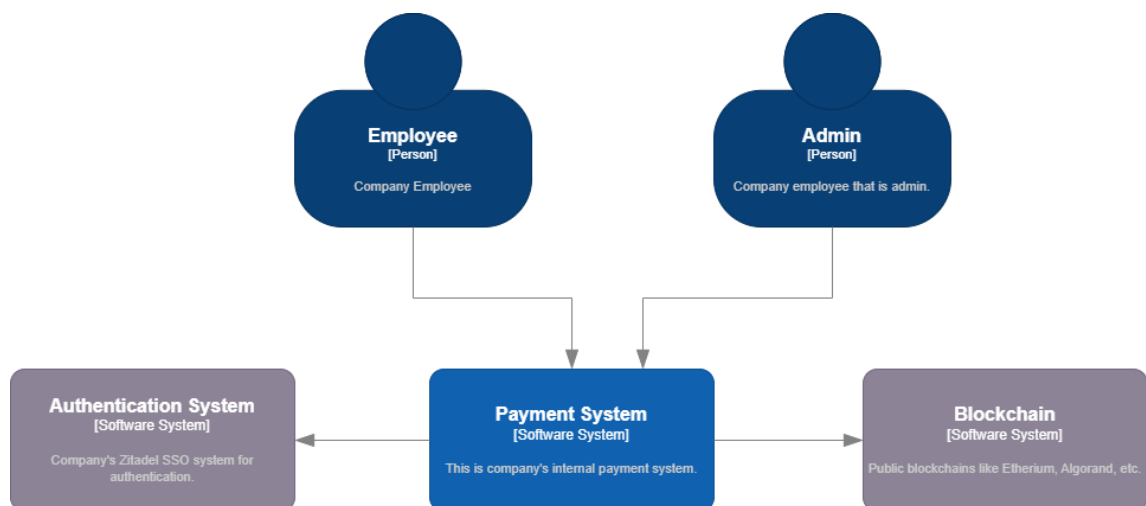
Zahtev	Opis
Z1.	Sistem za autentifikaciju i autorizaciju korisnika.
Z2.	Kreiranje i upravljanje novčanicima.
Z3.	Kreiranje i upravljanje tokenima.
Z4.	Verzionisanje tokena i rad sistema sa različitim verzijama tokena.
Z5.	Validacija tokena.
Z6.	Kreiranje različitih tipova transakcija, deponovanje sredstava, plaćanje i skidanje sredstava sa novčanika.
Z7.	Validacija transakcije.
Z8.	Upis dokaza o izvršetku transakcija u blokčejn.

4. Dizajn rešenja

Nakon definisanja zahteva koje rešenje treba da ispuni, u ovoj sekciji objašnjeno je na koji način su dizajnirani sistem i njegove komponente. Dizajn i arhitektura sistema biće predstavljen pomoću C4 modela koji je prethodno objašnjen u radu.

4.1. Kontekst

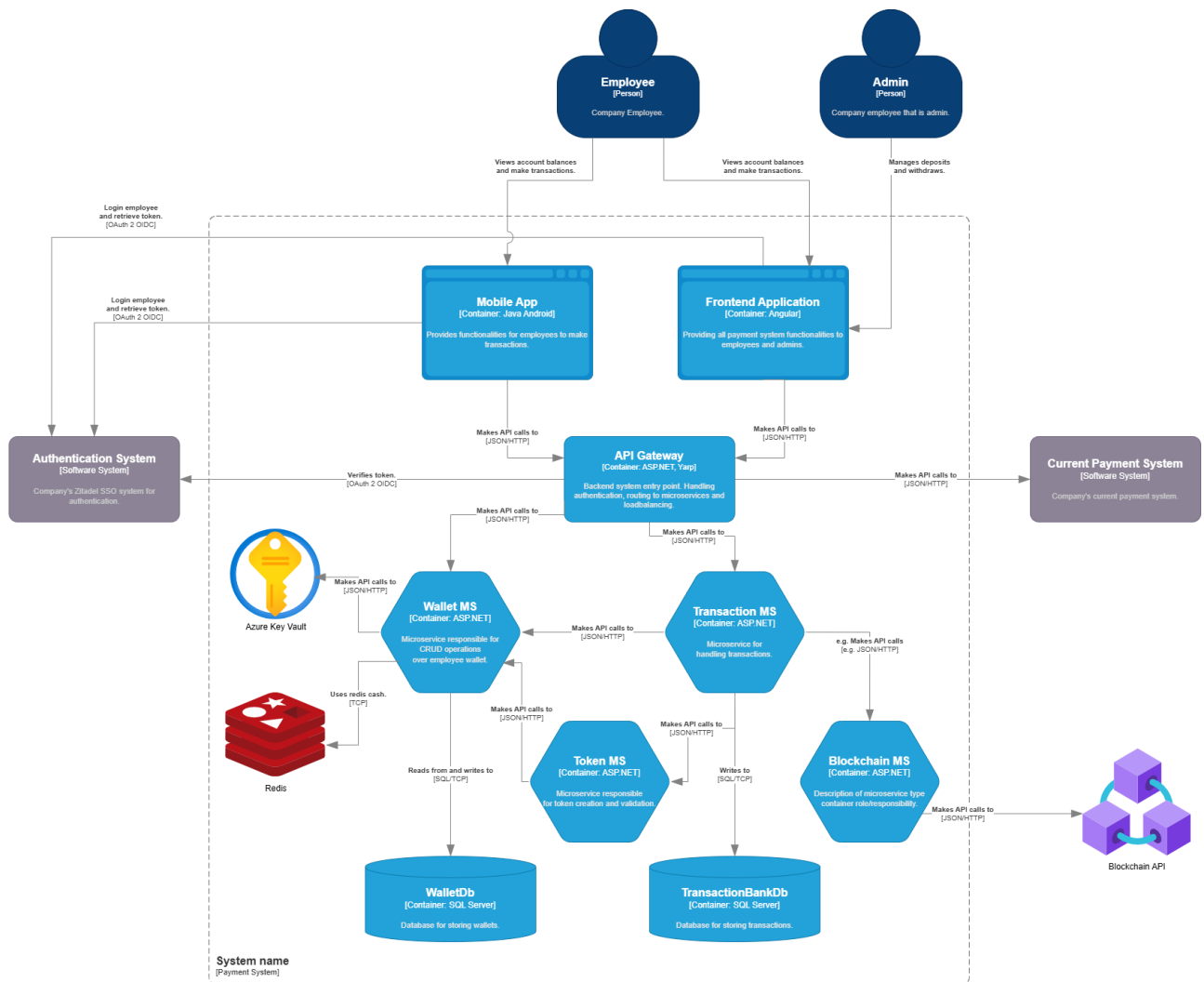
Na najapstraktnijem nivou na narednoj slici, prikazan je kontekst softverskog sistema dizajniranog tako da predstavlja rešenje definisanog problema. Postoje dve uloge u sistemu, zaposleni i admin. Zaposleni može da izvršava transakcije plaćanja, dok admin dodatno može da deponuje i skida sredstva iz novčanika zaposlenih. Oni direktno koriste zatvoreni sistem plaćanja koji komunicira sa sistemom za autentifikaciju (*Zitadel SSO*), kao i sa javnim blokčejnom za upis dokaza o izvršavanju transakcija unutar zatvorenog sistema plaćanja.



Slika 11 Kontekst dijagram.

4.2. Kontejneri

Na slici ispod prikazana je detaljnija arhitektura sistema za zatvoreno plaćanje odnosno kontejner dijagram C4 modela. Svaki kontejner na narednoj slici može se posmatrati kao zasebna aplikacija.



Slika 12 Arhitektura sistema zatvorenog plaćanja. Kontejner dijagram.

U narednoj tabeli može se videti pregled kontejnera u okviru celog sistema. Tabela sadrži tri kolone, naziv kontejnera, opis i tehnologije koje su korišćene u okviru kontejnera.

Tabela 2 Tabelarni prikaz kontejnera iz C4 modela.

Kontejner	Opis	Tehnolgije
<i>Mobile App</i>	Mobilna aplikacija namenjena zaposlenima da izvršavaju transakcije.	<i>Java Android</i>
<i>Frontend Application</i>	Veb aplikacija primarno namenjena zaposlenima i adminima da izvršavaju sve funkcionalnosti koje im sistem pruža	<i>Angular</i>
<i>API Gateway</i>	Ulazna tačka <i>backend</i> sistema koja obezbeđuje autentifikaciju i autorizaciju korisnika, rutiranje zahteva, kontrolu broja poslatih zahteva ka određenom mikroservisu i skaliranje.	ASP.NET, Yarp
<i>WalletMS</i>	Mikroservis zadužen za kreiranje i upravljanje digitalnim novčanicima.	ASP.NET, Entity Framework Core, Redis, Azure Key Vault

<i>TokenMS</i>	Mikrosevis zadužen za kreiranje, validaciju i verzionisanje tokena.	ASP.NET
<i>TransactionMS</i>	Mikroservis zadužen za kreiranje i upisivanje transakcija u bazu podataka.	ASP.NET, <i>Entity Framework Core</i>
<i>BlockchainMS</i>	Mirkoservis zadužen za upis dokaza o izvršavanju transakcija u neki od javno dostupnih blokčejnova	ASP.NET
<i>WalletDb</i>	Baza podataka zadužena za perzistenciju digitalnih novčanika i tokena.	<i>SQLite</i>
<i>TransactionBankDb</i>	Baza podataka zadužena za perzistenciju svih izvršenih transakcija.	<i>SQLite</i>

Sa klijentske strane nalaze se mobilna i veb aplikacija putem kojih korisnici koriste funkcionalnosti sistema. Ove dve aplikacije zajedno sa postojećim *backend* delom sistema nisu predmet rada već su dodate u dijagram kako bi se upotpunio prikaz o trenutnoj arhitekturi celokupnog sistema gde je zamišljeno da uporedo radi postojeći sistem za plaćanje kao i novi tokenizovani sistem zatvorenog plaćanja kako bi se uočile potencijalne razlike u tačnosti i performansama ova dva sistema. Da bi ovo bilo moguće u okviru postojećeg *backend* sistema izvršene su modifikacije sistema za autentifikaciju, odnosno integrisan je *Zitadel SSO* sistem koji se koristi i u novom sistemu.

API Gateway je kontejner koji predstavlja ulaznu tačku sistema iz spoljašnjeg sveta i razvijen je sa ciljem da obezbedi da se na jednom mestu vrši autentifikacija i autorizacija korisnika čime se ispunjava zahtev Z1. Pored toga njegova uloga jeste i rutiranje zahteva na odgovarajuće mikroservise, ograničavanje broja poziva ka nekom od mikroservisa u određenom vremenskom periodu (engl. *rate limiting*), kao i skaliranje određenih mikroservisa ukoliko se ukaže potreba za tim. Sve što se nalazi iza *API Gateway*-a zaštićeno je privatnom mrežom (engl. *private network*).

Glavni deo sistema u kontekstu funkcionalnosti koje sistem pruža jesu četiri mikroservisa od kojih svaki predstavlja zaseban kontejner. *WalletMS* mikroservis je zadužen za kreiranje digitalnih novčanika korisnika i upisivanje u *WalletDb* bazu podataka koja takođe predstavlja zaseban kontejner. *Azure Key Vault* služi za bezbedno čuvanje informacije o tome koji novčanik pripada kom korisniku, dok se Redis Cache koristi za zaključavanje digitalnih novčanika prilikom transakcije o čemu će biti više rečeno u sekciji gde je objašnjena implementacija rešenja. Ovim mikroservisom ispunjava se zahtev rešenja Z2. *TokenMS* mikroservis predstavlja mikroservis koji služi za kreiranje, validaciju i verzionisanje tokena čime se ispunjavaju zahtevi Z3, Z4 i Z5. *TransactionMS* je glavni mikroservis zadužen za kreiranje različitih tipova transakcija, deponovanje, plaćanje ili skidanje sredstava sa digitalnih novčanika i upisivanje isitih u *TransactionBankDb* čime se ispunjavaju zahtevi Z6 i Z7. *TransactionBankDb* je takođe poseban kontejner i to je baza podataka koja predstavlja jedinstveni izvor istine u okviru sistema gde se beleže sve izvršene transakcije. Četvrti mikroservis, *BlockchainMS* jeste mikroservis koji je zadužen za upis dokaza o izvršavanju transakcija u okviru zatvorenog sistema plaćanja u neki od javno dostupnih blokčejnova, na primer *Ethereum* čime je ispunjen zahtev Z8.

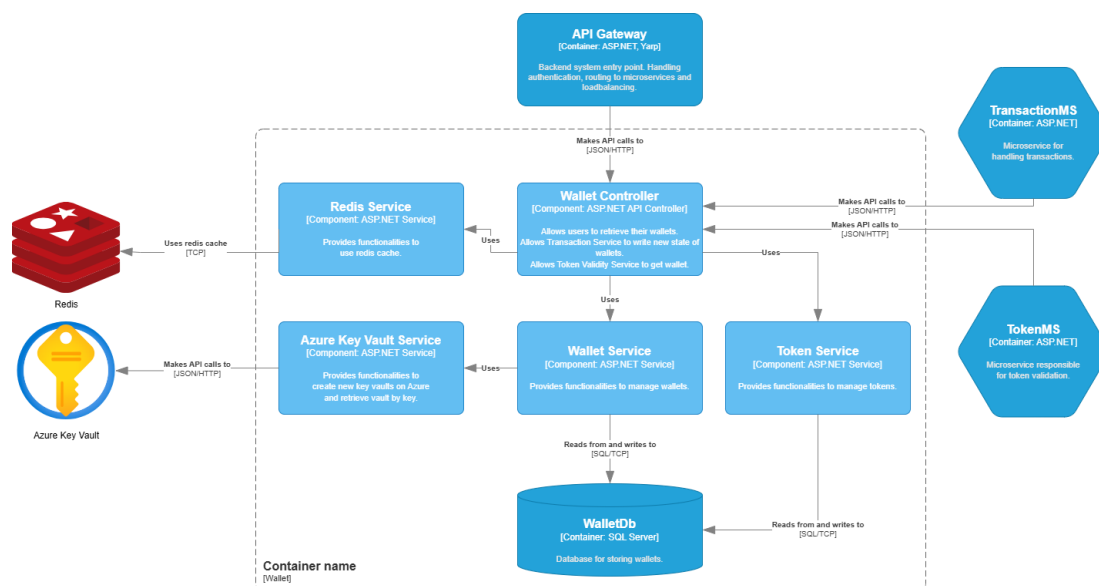
4.3. Komponente

U okviru ove sekcije biće prikazan detaljniji dizajn odnosno arhitektura četiri mikroservisa prethodno objašnjenih. U okviru svakog od navedenih kontejnera biće prikazano od kojih komponenti se sastoji, a nakon toga i objašnjeno čemu koja komponenta služi. Što se tiče *API Gateway*, *WalletDb* i *TransactionBankDb* kontejnera, dijagrami komponenti za iste nisu pravljani jer praktično jasno

definisane komponente u okviru tih kontejnera i nepostoje. Najbitniji delovi ova tri kontejnera biće prikazani u kod sekciji, kao i bitne i zanimljive metode mikroservisa.

4.3.1 *WalletMS*

Na narednoj slici prikazan je dijagram komponenti C4 modela za *WalletMS* kontejner.

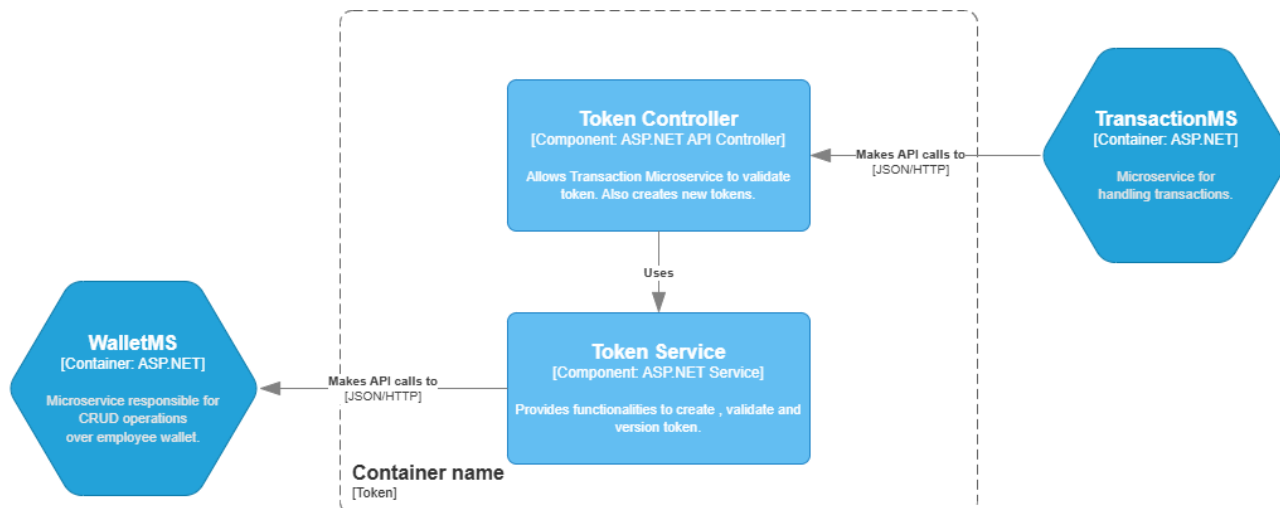


Slika 13 Komponent dijagram *WalletMS* kontejnera.

WalletMS je izgrađen od *Wallet Controller*-a koji obezbeđuje neophodne pristupne tačke za *API Gateway*, *TransactionMS* i *TokenMS*. Pored toga ovaj kontroler kao jedan od zadataka ima da uz pomoć funkcionalnosti *Redis Service*-a vodi brigu o zaključavanju određenih digitalnih novčanika dok traje izvršavanje transakcije. Pored kontrolera, *WalletMS* se sastoji od četiri servisa, *Redis servis*, *Azure Key Vault servis*, *Wallet servis* i *Token servis*. *Redis servis* zadužen je za upravljanje redis kešom i sadrži dve generičke metode *GetListAsync* i *StoreListAsync* koje omogućavaju dovljačenje i upisavanje podataka u redis keš memoriju. *Wallet servis* se sastoji od tri metode, *GetWallet*, *CreateWallet* i *GetWalletsForTransaction*. Prva navedena metoda služi za dovljačenje digitalnog novčanika ulogovanom korisniku ukoliko je novčanik već kreiran, dok druga služi za kreiranje novčanika ulogovanom korisniku. Obe metode koriste funkcionalnosti *Azure Key Vault* servisa kako bi dobavili Id digitalnog novčanika ili ga zapisali u *Key Vault* prilikom kreiranja novčanika. Metoda *GetWalletsForTransaction* služi za dovljačenje digitalnih novčanika i tokena koji im pripadaju prilikom izvršavanja transakcije. Ovaj servis komunicira sa *WalletDb* bazom podataka putem *Entity Framework Core*-a za perzistenciju podataka o digitalnim novčanicima. *Token servis* se sastoji od metoda *GetTokenForTransaction*, *GetTokensForTransaction* i *UpdateTokensByTransaction*. Ove metode služe za dovljačenje potrebnih tokena prilikom izvršavanja transakcije odnosno za ažuriranje tokena tj. upis novonastalih vrednosti polja tokena tokom transakcije. Ovaj servis takođe komunicira sa *WalletDb* bazom podataka kako bi perzistovao informacije o najnovijim validnim stanjima tokena.

4.3.2 *TokenMS*

Na narednoj slici prikazan je dijagram komponenti C4 modela za *TokenMS* kontejner.



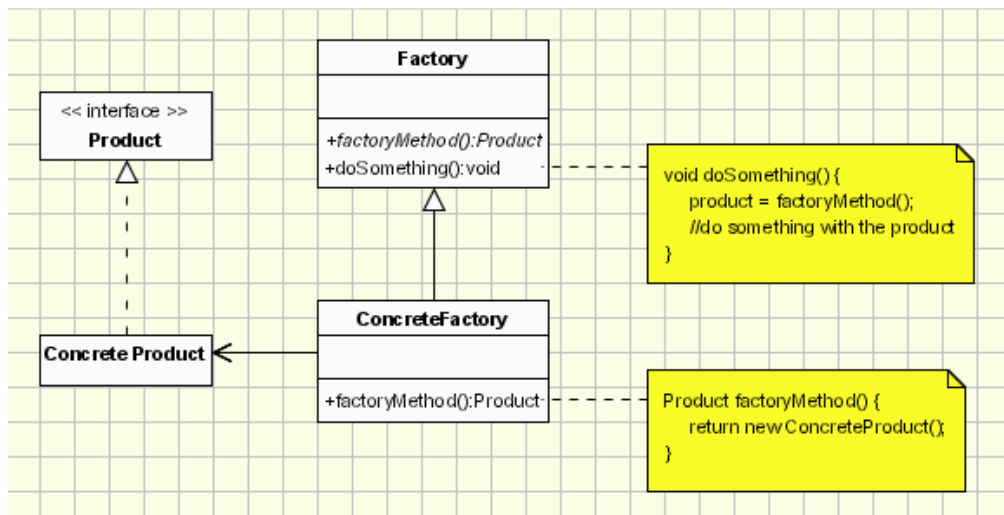
Slika 14 Komponent dijagram *TokenMS* kontejnera.

TokenMS je izgrađen od *Token Controller*-a koji obezbeđuje neophodne pristupne tačke za *TransactionMS*. *Token* servis se sastoji od dve metode, *CreateNewToken* i *ValidateToken* koje pružaju funkcionalnosti kreiranja novih tokena i validiranje postojećih. Ovaj mikroservis je bez čuvanja stanja (engl. *stateless*) i moguće ga je skalirati, odnosno podići više instanci servisa, radi postizanja boljih performansi. Takođe ovaj mikroservis implementira dizajnerski obrazac *Factory Method* za kreiranje instanci objekata tokena različitih verzija. U nastavku je objašnjen teorijski koncept ovog dizajnerskog obrasca.

Factory Method je kreacioni dizajnerski obrazac koji obezbeđuje interfejs za kreiranje objekata unutar nadklase, dok omogućava podklasama da promene tip objekata koji će biti kreirani. Ovaj obrazac je posebno koristan kada se tačan tip objekata koji treba kreirati može menjati ili mora biti određen u toku rada programa, čime se omogućava fleksibilnost i proširivost u procesu kreiranja objekata. [51]

Komponente *Factory Method* dizajnerskog obrasca su sledeće:

- **Kreator** - Ovo je apstraktna klasa ili interfejs koji deklarise fabrički metod. Kreator obično sadrži metodu koja služi kao fabrika za kreiranje objekata, a može sadržati i druge metode koje rade sa kreiranim objektima.
- **Konkretni Kreator** - Konkretna klase kreatora su podklase kreatora koje implementiraju fabrički metod za kreiranje specifičnih tipova objekata. Svaki konkretni kreator je odgovoran za kreiranje određenog proizvoda.
- **Proizvod** - Ovo je interfejs ili apstraktna klasa za objekte koje fabrički metod kreira. Proizvod definiše zajednički interfejs za sve objekte koje fabrički metod može da kreira.
- **Konkretni Proizvod** - Konkretna klase proizvoda su stvarni objekti koje fabrički metod kreira. Svaka konkretna klasa proizvoda implementira interfejs proizvoda ili proširuje apstraktnu klasu proizvoda.



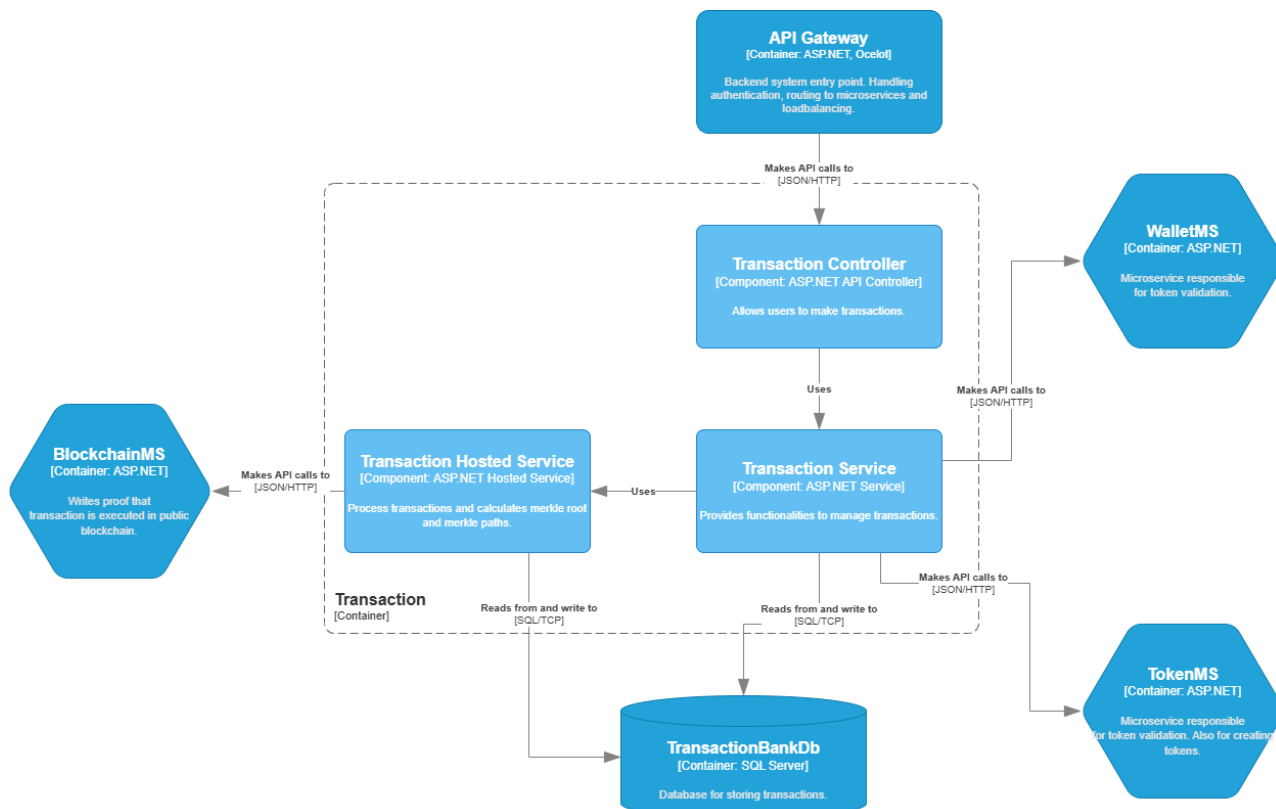
Slika 15 UML klasni dijagram *Factory Method* dizajnerskog obrazca. Preuzeto iz [51]

Factory Method obrazac treba koristiti kada je proces kreiranja objekata složen ili varira u zavisnosti od različitih uslova, jer može pojednostaviti kod i povećati mogućnost ponovne upotrebe. Ovaj obrazac omogućava kreiranje objekata putem interfejsa ili apstraktne klase, skrivajući detalje konkretnih implementacija. Na taj način smanjuje zavisnosti i olakšava modifikaciju ili proširenje sistema bez uticaja na postojeći kod. Ako aplikacija treba da kreira različite verzije proizvoda ili može da uvede nove tipove u budućnosti, *Factory Method* pruža fleksibilan način za upravljanje tim varijacijama definisanjem specifičnih fabričkih metoda za svaki tip proizvoda. Takođe, fabrike mogu inkapsulirati logiku konfiguracije, omogućavajući klijentima da prilagode proces kreiranja objekta putem parametara.

Prednosti *Factory Method* obrazca uključuju smanjeno povezivanje između kreatora i konkretnih proizvoda, što olakšava održavanje i proširenje sistema. Obrazac omogućava primenu principa jedinstvene odgovornosti (engl. *single responsibility principle*), jer se kod za kreiranje proizvoda može izdvojiti na jedno mesto u programu, čime se povećava preglednost i olakšava održivost koda. Takođe, omogućava primenu otvorenog-zatvorenog principa (engl. *open closed principle*), što znači da se novi tipovi proizvoda mogu dodati bez izmene postojećeg koda za druge proizvode. S druge strane, primena ovog obrasca može povećati složenost koda, jer zahteva uvođenje dodatnih klasa za svaku novu implementaciju proizvoda.

4.3.3 TransactionMS

Na narednoj slici prikazan je dijagram komponenti C4 modela za *TransactionMS* kontejner. *TransactionMS* je izgrađen od *Transaction Controller*-a koji obezbeđuje neophodne pristupne tačke za *API Gateway*. Sadrži tri pristupne tačke *CreateTransactionPayment*, *CreateTransactionDeposit* i *CreateTransactionWithdraw* koje pružaju funkcionalnosti kreiranja transakcija plaćanja, deponovanja i skidanja sredstava sa digitalnog novčanika. *Transaction* servis sadrži metode za kreiranje tokena koje pozivaju *TokenMS* koji sadrži logiku i odgovornost za kreiranje tokena i vraća kreiranje tokene, takođe sadrži metode za dobavljanje tokena koje pozivaju *WalletMS* koji sadrži logiku i odgovornost za dobavljanje tokena koji su *TransactionMS* mikroservisu neophodni za izvršavanje transakcije. Pored ovih metoda poseduje i metodu *CreateTransactionToProcess* koja pri iniciranju transakcije kreira objekat klase *TransactionToProcess* nakon čega se pozivom metode *AddTransactionToPool* dodaje transakcija u bazen transakcija (engl. *transaction pool*).

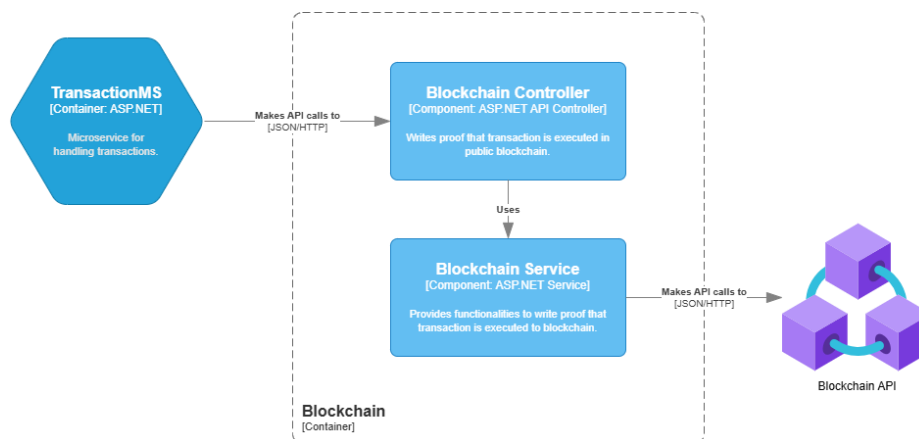


Slika 16 Komponent dijagram *TransactionMS* kontejnera.

Transaction Hosted Service je hostovani servis koji radi u pozadini i prikuplja transakcije za procesuiranje u određenom vremenskom periodu, nakon što taj period istekne poziva se metoda *ProcessTransactions* koja generiše Merkelovo stablo, validacione putanje za svaku transakciju koja se nalazi u bazenu transakcija. Nakon toga poziva se *BlockchainMS* koji ima zadatak da upiše merkelov koren u javni blokčejn i da vrati potvrdu o upisu.

4.3.4 BlockchainMS

Na narednoj slici prikazan je dijagram komponenti C4 modela za *BlockchainMS* kontejner.



Slika 17 Komponent dijagram *BlockchainMS* kontejnera.

BlockchainMS je izgrađen od *Blockchain Controller*-a koji obezbeđuje pristupnu tačku za *TransactionMS*. Takođe sadrži *Blockchain* servis koji sadrži metodu za upis Merkelovog korena na javni blokčejn.

5. Implementacija rešenja

Obzirom da je kompletan kod svih kontrolera i servisnih klasa kontejnera *WalletMS*, *TokenMS*, *TransactionMS* i *BlockchainMS* dat u prilogima ovog rada, u ovoj sekciji fokus će biti na komponentama za kreiranje tokena, heš funkcijama i algoritmima za generisanje korena Merkelovog stabla i verifikacionih putanja. Takođe prikazan je pametni ugovor (engl. smart contract) koji je neophodan za upis dokaza o izvršenju transakcija u *Ethereum* blokčejn.

5.1 Komponente za dobavljanje, kreiranje i ažuriranje tokena određene verzije

Za dobavljanje, kreiranje i ažuriranje tokena određene verzije koristi se dizajnerski obrazac *Factory Method* objašnjen prethodno u okviru teorijskih koncepata. U narednom listingu priložen je kod *IToken* interfejsa, to je interfejs koji je neophodno da implementira svaka nova klasa tokena koja predstavlja novu verziju tokena.

Listing 1 *IToken* interfejs.

```
public interface IToken
{
    bool IsTokenValid();
    TokenDto ToTokenDto();
}
```

Zatim u nastavku može se videti prva verzija tokena odnosno klasa *TokenV1* koja implementira prethodno navedeni interfejs.

Listing 2 *TokenV1* klasa.

```
public class TokenV1 : IToken
{
    private static readonly Func<byte[], int> HashFunction = HashFunctionV1.ComputeHash;
    public Guid TokenId;
    public int TokenIdHash { get; set; }
    public TokenDataV1 TokenData;
    public int TokenDataSignature { get; set; }
    public Guid WalletAddress { get; set; }
    public int WalletAddressHash { get; set; }
    public TokenV1(TokenDto token)
    {
        TokenId = token.TokenId;
        TokenIdHash = token.TokenIdHash;
        TokenDataSignature = token.TokenDataSignature;
        WalletAddress = token.WalletAddress;
        WalletAddressHash = token.WalletAddressHash;
        TokenData = JsonConvert.DeserializeObject<TokenDataV1>(token.TokenData);
    }
    public TokenV1(Guid walletAddress, decimal balance, Guid transactionId,
        TokenTypeEnum tokenType)
    {
        TokenId = Guid.NewGuid();
        WalletAddress = walletAddress;
        TokenData = new TokenDataV1()
        {
            Balance = balance,
            Version = "1.0",
            Created = DateTime.UtcNow,
            TokenTypeId = tokenType,
            TransactionId = transactionId
        };
        CalculateHashValues();
    }
    public bool IsTokenValid()
    {
        bool isValid = true;
        if (TokenDataSignature
            != HashFunction(CreateIntArrayForSignatureHashCalculation()
                .ToArray()))
        {
            isValid = false;
        }
        if (TokenIdHash != HashFunction(TokenId.ToArray()))
        {
            isValid = false;
        }
        if (WalletAddressHash != HashFunction(WalletAddress.ToArray()))
        {
            isValid = false;
        }
        if (TokenData.BalanceHash != HashFunction(TokenData.Balance.ToArray()))
        {
            isValid = false;
        }
        if (TokenData.VersionHash != HashFunction(TokenData.Version.ToArray()))
        {
            isValid = false;
        }
        if (TokenData.CreatedHash != HashFunction(TokenData.Created.ToArray()))
        {
            isValid = false;
        }
        if (TokenData.TokenTypeIdHash
            != HashFunction(TokenData.TokenTypeId.ToString().ToArray()))
        {
            isValid = false;
        }
        if (TokenData.TransactionIdHash
            != HashFunction(TokenData.TransactionId.ToArray()))
        {
            isValid = false;
        }
        return isValid;
    }
}
```

```

        public TokenDto ToTokenDto()
        {
            var tokenDto = new TokenDto()
            {
                TokenId = TokenId,
                TokenIdHash = TokenIdHash,
                TokenData = JsonConvert.SerializeObject(TokenData),
                TokenDataSignature = TokenDataSignature,
                WalletAddress = WalletAddress,
                WalletAddressHash = WalletAddressHash
            };
            return tokenDto;
        }
        private void CalculateHashValues()
        {
            TokenData.BalanceHash = HashFunction(TokenData.Balance.ToByteArray());
            TokenData.VersionHash = HashFunction(TokenData.Version.ToByteArray());
            TokenData.CreatedHash = HashFunction(TokenData.Created.ToByteArray());
            TokenData.TokenTypeIdHash =
                HashFunction(TokenData.TokenTypeId.ToString()
                    .ToByteArray());
            TokenData.TransactionIdHash =
                HashFunction(TokenData.TransactionId
                    .ToByteArray());
            TokenIdHash = HashFunction(TokenId.ToByteArray());
            WalletAddressHash = HashFunction(WalletAddress.ToByteArray());
            TokenDataSignature =
                HashFunction(CreateIntArrayForSignatureHashCalculation()
                    .ToByteArray());
        }
        private int[] CreateIntArrayForSignatureHashCalculation()
        {
            return [
                TokenIdHash,
                WalletAddressHash,
                TokenData.BalanceHash,
                TokenData.VersionHash,
                TokenData.CreatedHash,
                TokenData.TokenTypeIdHash,
                TokenData.TransactionIdHash
            ];
        }
    }
}

```

Ova klasa implementira metodu *IsTokenValid* koja validira token upotrebom heš funkcije koja se koristi za ovu verziju tokena. Ova heš funkcija predstavlja metod univerzalnog hešovanja objašnjen u okviru teorijskih koncepata i koristi se prilikom kreiranja tokena u pomoćnoj metodi *CalculateHashValues*.

Listing 3 *HashFunctionV1* klasa.

```

public static class HashFunctionV1
{
    public static int ComputeHash(byte[] input)
    {
        if (input == null || input.Length == 0)
            return 0;
        int hash = 17;
        foreach (byte b in input)
        {
            hash = (hash * 31 + b) % 23;
        }
        return hash;
    }
}

```

Takođe jedan od svojstava klase jeste *TokenData* koja predstavlja strukturu samog tokena, izdvojena je u posebnu klasu *TokenDataV1* koja se može videti u narednom listingu.

Listing 4 *TokenDataV1* klasa.

```
public class TokenDataV1
{
    public decimal Balance { get; set; }
    public int BalanceHash { get; set; }
    public string Version { get; set; }
    public int VersionHash { get; set; }
    public DateTime Created { get; set; }
    public int CreatedHash { get; set; }
    public TokenTypeEnum TokenTypeId { get; set; }
    public int TokenTypeIdHash { get; set; }
    public Guid TransactionId { get; set; }
    public int TransactionIdHash { get; set; }
    public string TokenProof { get; set; }
}
```

Na samom kraju, stižemo do klase *TokenFactory* koja obezbeđuje funkcionalnosti dobavljanja, kreiranja i ažuriranja tokena. Metoda *GetToken* služi za dobavljanje instance klase tokena određene verzije, dok metode *CreateToken* i *UpdateToken* treba uvek da vrate instancu klase poslednje verzije tokena, što je odgovornost developera da izmeni u okviru ovih metoda prilikom kreiranja nove klase odnosno nove verzije tokena.

Listing 5 *TokenFactory* klasa.

```
public class TokenFactory
{
    public IToken GetToken(string version, TokenDto token)
    {
        switch (version)
        {
            case "1.0":
                return new TokenV1(token);
            default:
                throw new Exception();
        }
    }
    public IToken CreateToken(Guid walletAddress,
        decimal amount, Guid transactionId, TransactionTypeEnum transactionType, TokenTypeEnum
        tokenType)
    {
        if (transactionType == TransactionTypeEnum.Withdraw)
        {
            throw new Exception("Token creation is not possible for withdraw transaction type.");
        }
        return new TokenV1(walletAddress, amount, transactionId, tokenType);
    }
    public IToken UpdateToken(TokenDto tokenDto,
        TransactionTypeEnum transactionType, decimal amount, bool? isFrom)
    {
        var token = new TokenV1(tokenDto);
        if (transactionType == TransactionTypeEnum.Deposit ||
            (transactionType == TransactionTypeEnum.Payment && isFrom == false))
        {
            token.TokenData.Balance += amount;
        }
        if (transactionType == TransactionTypeEnum.Withdraw ||
            (transactionType == TransactionTypeEnum.Payment && isFrom == true))
        {
            token.TokenData.Balance -= amount;
        }
        return token;
    }
}
```


5.2 Komponente za generisanje Merkelovog korena i validacionih putanja

Za kreiranje Merkelovog korena i validacionih putanja kreirana je *MerkleRoot* statička klasa u okviru *TransactionMS* mikroservisa koja sadži *GenerateMerkleRoot* metodu za generisanje korena Merkelovog stabla i *GetMerklePath* metodu za generisanje validacionih putanja za svaku od transakcija koja ulazi u proces generisanja merkelovog korena. U narednom listingu može se videti implementacija ovih metoda odnosno *MerkleRoot* klase.

Listing 6 *MerkleRoot* klasa.

```
public static class MerkleRoot
{
    private readonly static Func<string,string> HashFunc = HashFunctions.ComputeSha256Hash;
    public static string GenerateMerkleRoot(List<string> transactions)
    {
        if (transactions.Count == 0) return string.Empty;
        List<string> currentLevel = new List<string>();
        foreach (var transaction in transactions)
        {
            currentLevel.Add(HashFunc(transaction));
        }
        while (currentLevel.Count > 1)
        {
            List<string> nextLevel = new List<string>();
            for (int i = 0; i < currentLevel.Count; i += 2)
            {
                if (i + 1 < currentLevel.Count)
                {
                    nextLevel.Add(HashFunc(currentLevel[i] + currentLevel[i + 1]));
                }
                else
                {
                    nextLevel.Add(currentLevel[i]);
                }
            }
            currentLevel = nextLevel;
        }
        return currentLevel[0];
    }
    public static List<string> GetMerklePath(List<string> transactions,
        string transaction, string merkleRoot)
    {
        return GenerateMerklePath(transactions, transaction, merkleRoot);
    }
    private static List<string> GenerateMerklePath(List<string> transactions,
        string targetTransaction, string merkleRoot)
    {
        List<string> merklePath = new List<string>();
        if (transactions.Count == 0)
            return merklePath;
        List<string> currentLevel = new List<string>();
        foreach (var transaction in transactions)
        {
            currentLevel.Add(HashFunc(transaction));
        }
        while (currentLevel.Count > 1)
        {
            List<string> nextLevel = new List<string>();
            for (int i = 0; i < currentLevel.Count; i += 2)
            {
                if (i + 1 < currentLevel.Count)
                {
                    string combinedHash = HashFunc(currentLevel[i]
                        + currentLevel[i + 1]);
                    nextLevel.Add(combinedHash);
                    if (currentLevel[i] == HashFunc(targetTransaction) ||
                        (i + 1 < currentLevel.Count &&
                            currentLevel[i + 1] == HashFunc(targetTransaction)))
                    {
                        // Add the sibling hash to the path
                        merklePath.Add(i + 1 < currentLevel.Count ?
                            currentLevel[i + 1] : currentLevel[i]);
                    }
                }
            }
        }
    }
}
```

```

        }
        else
        {
            nextLevel.Add(currentLevel[i]);
        }
    }
    currentLevel = nextLevel;
}
merklePath.Add(merkleRoot);
return merklePath;
}
}

```

Ove metode koriste SHA-256 algoritam za hešovanje čvorova Merkelovog stabla prilikom kreiranja korena i validacionih putanja. Koristi se ugrađen algoritam koji se nalazi u *System.Security.Cryptography* biblioteci .NET okvira, s tim što je obavijen u posebnu metodu *ComputeSha256Hash* i izdvojen u posebnu statičnu klasu *HashFunctions* kako bi bio lakše zamenljiv u klasi *MerkleRoot* gde je dodat kao properti *HashFunc* odnosno delegat što se može videti u listingu iznad. Delegat predstavlja referencu na metodu.

Listing 7 *HashFunctions* klasa.

```

public static class HashFunctions
{
    public static string ComputeSha256Hash(string rawData)
    {
        using (SHA256 sha256Hash = SHA256.Create())
        {
            var hash = sha256Hash.ComputeHash(Encoding.UTF8.GetBytes(rawData));
            return BitConverter.ToString(hash).Replace("-", "").ToLower();
        }
    }
}

```

5.3 Pametni ugovor za upis u *Ethereum* blokčejn

Za upis dokaza o izvršavanju transakcija u *Ethereum* blokčejn neophodno je kreirati pametni ugovor. Pametni ugovor je napisan programskim jezikom *Solidity*, jezik namenjen za pisanje pametnih ugovora, pomoću *RemixIDE* okruženja. Ovo okruženje omogućava pisanje pametnog ugovora, njegovo *build*-ovanje kao i *deploy*-ovanje. Kreirani pametni ugovor, nakon *build*-a, *deploy*-ovan je na *Sepolia* mrežu pomoću *Infura* veb-platforme. *Sepolia* je testni *Ethereum* blokčejn, odnosno njegova replika, koja omogućava testiranje pametnih ugovora i ostalih funkcionalnosti koje pruža *Ethereum* blokčejn. *MetaMask* je digitalni novčanik za kriptovalute koji omogućava povezivanje sa različitim novčanicima određenih kriptovaluta. Kreiran je nalog na *MetaMask*-u za *SepoliaETH* kriptovalutu, koji je zatim uvezan na *Infura* platformi kako bi se mogao koristiti prilikom testiranja pametnog ugovora na *Sepolia* mreži. Zatim je preko *Google Cloud Web3* veb-aplikacije, aplikacija koja omogućava besplatan depozit testnih kriptovaluta, na *MetaMask* novčanik dodato 0.5 *SepoliaETH* za potrebe testiranja. U sledećem listingu može se videti kreirani pametni ugovor za upis dokaza o izvršavanju transakcija.

Listing 8 Pametni ugovor za upis dokaza o izvršavanju transakcija

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.0;

contract SimpleStorage {
    string public storedData;

    function set(string memory x) public {
        storedData = x;
    }

    function get() public view returns (string memory) {
        return storedData;
    }
}
```

Pametni ugovor sadrži samo jedno polje *storedData* i funkcije *get* i *set*. Funkcija *get* služi za vraćanje vrednosti tog polja, dok *set* služi za setovanje vrednosti, odnosno ovom funkcijom se postavlja vrednost ovog polja na vrednost korena Merkelovog stabla nastalog izvršavanjem transakcija. Zatim se ova vrednost upisuje prilikom izvršavanje blokčejn transakcije kako bi se upisao dokaz u sam blokčejn. Ovaj pametni ugovor odnosno metoda *set* se poziva iz *BlockchainMS* mikroservisa upotrebom *Netherium* biblioteke, biblioteka koja pruža funkcionalnosti za interakciju sa pametnim ugovorima i *Ethereum* klijentima. Kompletan kod kontrolera i servisne klase ovog mikrosevisa može se videti u prilogu rada.

6. Evaluacija rešenja nad *ZeroCash* sistemom

ZeroCash predstavlja postojeći tradicionalni sistem zatvorenog plaćanja koji ne koristi koncepte kao što su tokenizacija i blokčejn tehnologija. Kao evaluacija rešenja biće poreden ovaj sistem sa implementiranim sistemom gde će se porediti stanja digitalnih novčanika ovog rešenja sa stanjima korisnikovih sredstava u *ZeroCash* sistemu nakon izvršavanja identičnih transakcija u oba sistema. Takođe, biće prikazan kratak uvid u performanse i bezbednost oba sistema. Na slici ispod može se videti arhitektura *ZeroCash* sistema.

Tabela 3 Redosled izvršenih transakcija.

Transakcija	Tip transakcije	Vrednost	Od (engl. <i>From</i>)	Za (engl. <i>To</i>)
T1	<i>deposit</i>	1000.37	/	282841169384177670
T2	<i>deposit</i>	2000.51	/	282841168260104198
T3	<i>payment</i>	100.13	282841169384177670	282841168260104198
T4	<i>withdraw</i>	253.78	282841168260104198	/

Pre izvršavanja transakcija stanja sredstava korisnika su bila nula. Nakon izvršenih transakcija u *ZeroCash* sistemu zabeleženo je sledeće stanje sredstava korisnika koje se može videti na narednoj slici.

	user character varying (30) 	balance numeric (20,2) 
1	282841168260104198	846.72
2	282841169384177670	1900.38

Slika 18 Stanje sredstava korisnika u *ZeroCash* sistemu nakon izvršenih transakcija.

Bitno je napomenuti da se sredstva u okviru *ZeroCash* sistema i token tipa *ZeroCash* u sistemu implementiranog rešenja posmatraju da imaju jednake vrednosti. Nakon izvršavanja transakcija u implementiranom rešenju zabeleženo je sledeće stanje korisnikovih sredstava koje se može videti na sledećoj slici.

Token	User wallet	Balance
D327BBA4-0F0D-4772-A583-27CA409287CF	B12B1C8C-A1ED-4084-BB4A-AFFAD1F95A6B	{ "Balance": 846.72, '}
0C3DCD3F-8795-42A0-A87C-0A06DD12BF56	7F975BF0-FEBD-4226-8025-DD9E38506FEA	{ "Balance": 1900.38, '}

Slika 19 Stanje sredstava korisnika u implementiranom rešenju nakon izvršavanja transakcija.

Korisniku čiji je ID 282841169384177670, odgovara digitalni novčanik čiji je ID 7F975BF0-FEBD-4226-8025-DD9E38506FEA, dok korisniku čiji je ID 282841168260104198 odgovara digitalni novčanik čiji je ID B12B1C8C-A1ED-4084-BB4A-AFFAD1F95A6B. Na osnovu toga može se zaključiti da su vrednosti korisnikovih sredstava jednake u oba sistema, te da implementirano rešenje ispunjava očekivanje tačnosti i konzistentnosti podataka. Takođe, može se videti da u implementiranom sistemu na nivou prezistencije podataka u bazi podataka, sredstva se čuvaju na nivou digitalnog novčanika i nije dostupna informacija o tome kojem korisniku pripada taj novčanik što svakako predstavlja bezbednije i sigurnije rešenje od načina perzistencije u okviru *ZeroCash* sistema. Kao dodatnu sigurnost dokaz o izvršavanju transakcija upisan je u blokčejn što se može videti na slici ispod na kojoj se nalaze upisane blokčejn transakcije. Upisane blokčejn transakcije se mogu pronaći putem *Etherscan* veb-aplikacije.

Transaction Hash	Method	Block	Date Time (UTC)	From	To	Amount	Txn Fee
0x321eea4ac9...	Transfer*	7037096	2024-11-08 15:12:48	0xe66D1d3B...42D4Cd21B	OUT 0x12345678...f12345678	0 ETH	0.00005111
0x3c78cfa93d1...	Transfer*	7037092	2024-11-08 15:12:00	0xe66D1d3B...42D4Cd21B	OUT 0x12345678...f12345678	0 ETH	0.00005958
0x04d98ec19a...	Transfer*	7037088	2024-11-08 15:11:00	0xe66D1d3B...42D4Cd21B	OUT 0x12345678...f12345678	0 ETH	0.00005294
0x8acb403962...	Transfer*	7037085	2024-11-08 15:10:00	0xe66D1d3B...42D4Cd21B	OUT 0x12345678...f12345678	0 ETH	0.00005179

Slika 20 Dokaz o upisanim transakcijama u *Ethereum* blokčejn.

Takođe, u bazi podataka u kojoj se čuvaju sve transakcije u okviru implementiranog rešenja upoređene su vrednosti *EthereumProof*-a sa vrednoću *TransactionHash*-a blokčejn transakcija gde se vidi da su ove vrednosti poklapaju odnosno da upis dokaza ispunjava očekivanja. Na narednoj slici mpože se videti stanje u bazi transakcija u okviru implementiranog rešenja.

Transaction Id	Ethereum Proof
5C29FDE5-ED2C-44A3-825F-330D3E65A08A	0x8acb40396211f3704196b05cfde9e983a7f4f82efc31f4e046c0bb7db9f9be07d
5565AA4A-115B-4F3D-B9D0-0AEB915BF157	0x04d98ec19aab1aba221135794f55145f19d86d28341459da35fb8fc43e25139e
71AD57E4-AE94-4849-9F24-CCA85DF8214D	0x3c78cfa93d19a56a2a23e9d2fab377263f58234fda55e9b891d28c50d1263c1c
415D939E-E6F7-4DE4-B749-7D78F0D3C7B6	0x321eea4ac97261c17dfc9fb10c3131634440fd284e840fad24257482fea5ab33

Slika 21 Stanje u bazi transakcija implementiranog rešenja nakon izvršavanja transakcija.

Bitno je napomenuti da je u kontekstu performansi primećemo da *ZeroCash* sistem za primer iznad radi brže od implementiranog rešenja, ali implementirano rešenje obezbeđuje daleko sigurniji i bezbedniji u kontekstu podataka korisnika i transakcija. Takođe, obzirom da su oba sistema pokrenuta na lokalnom okruženju i da se radi o malom broju transakcija, moguće je očekivati bolje performanse implementiranog rešenja za veći broj transakcija zbog mogućnosti skaliranja mikroservisa i načina rada implementiranog Merkelovog stabla objašnjenog prethodno u radu.

7. Diskusija

Nakon uvoda i prikaza teorijskih koncepata vezanih za tokenizaciju zatvorenih sistema plaćanja, definisani su zahtevi za kreiranje pokaznog primera primenjujući određene koncepte, a zatim je taj primer i implementiran. Tokom same implementacije stečena su određena nova znanja o algoritmima i strukturama podataka za tokenizaciju, i primećene su određene prednosti kao i ograničenja ovakvog rešenja koja će biti navedena u nastavku ove sekcije.

Tokom implementacije *BlockchainMS*-a, odnosno upisa dokaza o izvršavanju transakcija u *Ethereum* blokčejn, primećeno je da upis u ovaj blokčejn nije besplatan. Svaki upis zahteva određenu količinu *gas*-a koji nije fiksiran već se stalno varira. Nakon uvida u ovakvo ponašanje *Ethereum* blokčejna, transakcione takse u okviru implementiranog rešenja mogu se konfigurisati tako da pokriju ove troškove upisa dokaza o izvršavanju transakcija u blokčejn. U toku implementacije ovog rešenja prilikom jednog upisa u blokčejn bilo je neophodno izdvojiti oko 0.35 američkih dolara. Takođe, obzirom da se kao dokaz o izvršavanju transakcija upisuje vrednost korena Merkelovog stabla nastalog od onoliko transakcija koliko ih se dogodilo u određenom vremenskom periodu, interesantno je prokomentarisati da porastom frekvencije transakcija u sistemu posmatrajući definisan vremenski period za prikupljanje transakcija koje kreiraju Merkelovo stablo, opada količina novčanih sredstava koja je potrebna za upis dokaza u blokčejn po transakciji.

7.1. Ograničenja

Kao jedno od glavnih ograničenja ovog rešenja jeste oslanjanje na aktivni direktorijum (engl. *active directory*) kao deo infrastrukture zadužen za autentifikaciju i kontrolu pristupa korisnika, što praktično znači da sistem koji želi da integriše ovo rešenje kao zatvoreni platni sistem u okviru svog sistema mora da poseduje svoj aktivni direktorijum. Ovo rešenje zahteva tehničku ekspertizu te potencijalno može povećati složenost održavanja sistema. Aktivni direktorijumi su neretko specifični za jednu organizaciju, što može ograničiti pristup i funkcionalnost za korisnike koji nemaju nalog u aktivnom direktorijumu ili nisu deo mreže što smanjuje celokupnu fleksibilnost sistema u tom pogledu. Glavna mana korišćenja aktivnog direktorijuma jeste to što ukoliko dođe do problema sa aktivnim direktorijumom ceo sistem plaćanja postaje nedostupan. Naravno korišćenje aktivnog direktorijuma za autentifikaciju i kontrolu pristupa korisnika ima i značajne prednosti koje će u nastavku rada biti naglašene.

Unos novca u zatvoreni sistem plaćanja, koji zahteva fizički kontakt sa administratorom, predstavlja značajno ograničenje jer ograničava skalabilnost i efikasnost sistema. Fizičko deponovanje sredstava kod administratora povećava zavisnost od administrativnih resursa, pa se pri većem broju transakcija javlja potreba za dodatnim osobljem koje upravlja procesom. Ovo ne samo

da produžava vreme potrebno za obavljanje depozita, već ograničava korisnike na radno vreme administratora, što može biti neadekvatno za korisnike sa hitnim potrebama za deponovanjem sredstava.

Metoda *GenerateMerkleRoot* klase *MerkleRoot* koja nije ograničena na fiksni broj transakcija koje ulaze u kreiranje merkelovog stabla, može predstavljati izazov ukoliko je velika frekvencija transakcija u kratkom vremenskom periodu, jer ukoliko broj transakcija nije ograničen može doći do kreiranja prevelikih stabala što direktno utiče na smanjenje performansi algoritma za verifikaciju validnosti transakcija. Za sisteme sa velikim brojem transakcija u kratkom vremenskom periodu ova metoda može se pokazati kao neoptimalna, gde je stabilnost performansi ključna. Prema tome ukoliko bi takvi sistemi želeli da integrišu ovo rešenje ovo je definitivno jedno od ključnih ograničenja u tom kontekstu.

Sve prethodno navedeno zajedno utiče na performanse sistema, pri čemu se brzina obrade transakcija može smanjiti zbog opterećenja prilikom generisanja Merkelovog stabla i validacionih putanja za svaku transakciju kada je broj transakcija velik. Skalabilnost i dostupnost sistema je ograničena oslanjanjem na fizičke procedure, poput ručnog deponovanja sredstava i povezivanja sa aktivnim direktorijumom, što može otežati prilagođavanje sistema povećanim zahtevima korisnika. Takođe ručno deponovanje sredstava dodatno povećavaju mogućnost ljudskih grešaka što može dovesti do neželjenih stanja na digitalnim novčanicima korisnika.

Bitno je napomenuti da je mikroservise *TokenMS* i *BlockchainMS* jednostavno skalirati jer su ti mikroservisi *stateless* te kreiranje više instanci ovih mikroservisa ne predstavlja problem, dok su mikroservisi *WalletMS* i *TransactionMS* *stateful* i njihovo skaliranje, odnosno kreiranje više instanci, zahteva naprednije tehnike skaliranja i čuvanja njihovih stanja što ograničava performanse ova dva mikroservisa.

Zatvoreni sistem plaćanja sa prethodno navedenim ograničenjima najbolje funkcionišu u manjim, kontrolisanim okruženjima sa ograničenom učestalošću transakcija. Za okruženja sa visokim zahtevima za frekvencijom i performansama, potrebna su optimizovana rešenja poput automatizovanog deponovanja, optimizacije struktura podataka Merkelovog stabla i potencijalno decentralizovanje metoda autentifikacije.

7.2. Prednosti

Kao glavna prednost ovog rešenja svakako jeste povećana bezbednost transakcija i korisnikovih sredstava upravo tokenizacijom i upisom dokaza o izvršavanju transakcija u javni blokčejn. U pogledu bezbednosti, kombinacija tokenizacije, upisa u blokčejn i mehanizma za zaključavanje novčanika prilikom izvršavanja transakcije pruža visoku otpornost na potencijalne pretnje, kao što su dvostruko trošenje (engl. *double spending*) i neovlašćeno menjanje transakcija što povećava transparentnost i integritet celokupnog sistema jer svaki blokčejn unosi podatke u decentralizovani, nepromenljivi registar, čime se obezbeđuje otpornost na manipulacije.

Niske transakcione takse su još jedna važna prednost ovog sistema sa stanovišta korisnika, jer su troškovi održavanja transakcija u zatvorenom okruženju niži nego u otvorenim sistemima plaćanja, gde pružaoci usluga naplaćuju veće naknade za izvršavanje transakcije od ovog rešenja. Ovo rešenje takođe pruža potpunu fleksibilnost u kontekstu transakcionih taksi jer se one mogu podesiti putem parametra na odgovarajući procenat vrednosti transakcije. Problem kod otvorenih sistema plaćanja jesu fiksne transakcione takse koje značajno utiču na transakcije male vrednosti što ovo rešenje eliminiše upravo upotrebom parametra za definisanje procenta transakcione takse što omogućava da su te takse direktno srazmerne za sve transakcije odnosno za bilo koju vrednost transakcije transakciona taksa će procentualno gledano biti jednaka.

Prethodno u ograničenjima sistema aktivni direktorijum je naglašen u kontekstu smanjenja fleksibilnosti i dostupnosti sistema, što se kompenzuje velikom prednošću u kontekstu sigurnosti i

bezbednosti sistema. Korišćenje aktivnog direktorijuma kao mehanizma za kontrolu pristupa u zatvorenom sistemu plaćanja donosi značajne prednosti u pogledu sigurnosti i upravljanja korisnicima, omogućavajući da samo ovlašćeni korisnici, koji su deo direktorijuma, mogu pristupiti sistemu, čime se minimizira rizik od neovlašćenog pristupa. Sistem automatski prepoznaje korisničke naloge, proverava njihove privilegije i dodeljuje odgovarajuće dozvole, olakšavajući upravljanje korisničkim pravima i osiguravajući pristup isključivo korisnicima sa validnim nalogima u aktivnom direktorijumu. Dodatna prednost aktivnog direktorijuma se ogleda u tome što se lako implementiraju politike zaštite, kao što su kompleksnost i redovno resetovanje lozinki, kao i višefaktorska autentifikacija, čime se dodatno unapređuje sigurnost sistema. Oslanjanjem na aktivni direktorijum postiže se maksimalna kontrola nad pristupom i smanjenje rizika od curenja poverljivih informacija.

Na kraju kao jedna od velikih prednosti ovo rešenja jeste potpuna kontrola nad sistemom, budući da je zatvoren i nije zavistan od spoljašnjih sistema plaćanja, omogućava se lakše prilagođavanje poslovnim potrebama, pravovremeno reagovanje na bezbednosne izazove i optimizacija troškova bez zavisnosti od regulativa koje često nameću otvorene sistemi plaćanja.

Ovo rešenje sa prethodno naglašenim prednostima pruža sistem koji je bezbedan i idealan za organizacije koje zahtevaju sigurno, centralizovano upravljanje i nadzor nad transakcijama, uz minimalnu zavisnost od eksternih pružaoca usluga.

8. Zaključak

Cilj ovog rada bio je istražiti mogućnosti implementacije tokenizacije sistema plaćanja zatvorene petlje upotrebom blokčejn tehnologije, digitalnih novčanika i tokena, u cilju poboljšanja bezbednosti transakcija i zaštite podataka. U uvodnim delovima rada predstavljene su osnove zatvorenih i otvorenih sistema plaćanja, kao i ključne karakteristike i prednosti zatvorenih sistema u smislu kontrole troškova, lojalnosti korisnika i smanjenja transakcionih troškova.

Nakon toga, prikazana je struktura digitalnih novčanika i različite implementacije zatvorenih sistema plaćanja na globalnom i lokalnom nivou, kao i teorijski koncepti neophodni za implementaciju ovog rešenja. Definisani su zahtevi rešenja koje je potrebno implementirati za postizanje zadatog cilja.

Implementacija zahteva rešenja obuhvatila je razvoj tokenizacije sistema zatvorenog plaćanja kroz upotrebu mikroservisne arhitekture gde se kroz različite servise primenila implementacija različitih heš funkcija, modifikacija Merkelovog stabla i upotreba blokčejn tehnologije kako bi se ispunili definisani zahtevi rešenja. Tokom implementacije, identifikovane su ključne prednosti kao što su veća sigurnost podataka, smanjenje rizika od povrede podataka i efikasnija kontrola pristupa. Prikazane su i glavne razlike ovakvog rešenja u odnosu na tradicionalne otvorene sisteme plaćanja, kao i ograničenja ovog sistema.

Na osnovu implementiranog rešenja, zaključeno je da zatvoreni sistemi plaćanja, uz implementaciju tokenizacije i blokčejn tehnologije, mogu značajno doprineti unapređenju sigurnosti i smanjenju troškova transakcija. Implementacija tokenizacije uz upotrebu blokčejn tehnologije omogućila je zaštitu osetljivih podataka korisnika, kao i poboljšanu transparentnost i integritet celokupnog sistema, dok je korišćenje aktivnog direktorijuma olakšalo kontrolu pristupa, upravljanje korisnicima u sistemu i dodatno poboljšalo sigurnost i bezbednost sistema.

Predlozi za unapređenje ovog rešenja obuhvataju nekoliko ključnih rešenja koja bi značajno povećala njegovu efikasnost, dostupnost i fleksibilnost. Prvo, implementacija mogućnosti upisa dokaza o izvršavanju transakcija u više javnih blokčejnova omogućila bi dodatnu sigurnost i transparentnost transakcija, kao i smanjenje rizika od gubitka podataka, čime bi se poboljšala otpornost sistema. Takođe, kao jedan od predloga unapređenja jeste implementacija zaključavanja samo tipova tokena koji učestvuju u transakciji, a ne kompletnih digitalnih novčanika korisnika, što bi omogućilo veću fleksibilnost i dostupnost sistema, omogućavajući korisnicima da u kraćem vremenskom periodu izvršavaju više transakcija sa različitim tipovima tokena. Implementacija otvorenog bankarstva za uplatu sredstava na digitalni novčanik predstavljala bi još jedno značajno unapređenje, jer bi se korisnicima omogućila brže i jednostavnije deponovanje sredstava bez potrebe za fizičkim uplaćivanjem istih kod administratora, čime bi se otklonio i faktor ljudske greške. Osim toga, uvođenje menjačnice unutar sistema za razmenu različitih tipova tokena omogućilo bi korisnicima konverziju sredstava i korišćenje raznovrsnijih usluga unutar ekosistema. Ova rešenja

doprinela bi unapređenju korisničkog iskustva, povećanju sigurnosti, dostupnosti i fleksibilnosti sistema, te omogućila jednostavnije korišćenje celokupnog sistema u svakodnevnim transakcijama.

LITERATURA

- [1] K. Radage, „An Overview of the Payments Ecosystem in 2024“, Credit Card Processing and Merchant Account. Приступљено: 29. Октобар 2024. [На Интернету]. Available at: <https://www.clearlypayments.com/blog/an-overview-of-the-payments-ecosystem-in-2024/>
- [2] T. text provides general information S. assumes no liability for the information given being complete or correct D. to varying update cycles и S. C. D. M. up-to-D. D. T. R. in the Text, „Topic: Open banking“, Statista. Приступљено: 08. Новембар 2024. [На Интернету]. Available at: <https://www.statista.com/topics/11010/open-banking/>
- [3] „Number of open banking API calls forecast 2027“, Statista. Приступљено: 08. Новембар 2024. [На Интернету]. Available at: <https://www.statista.com/statistics/1389414/number-of-open-banking-api-calls-worldwide/>
- [4] A. M. Alex Malyshev, „Closed Loop Payment Systems: Definition, Benefits and Business Cases“. [На Интернету]. Available at: <https://sdk.finance/closed-loop-payment-systems-definition-benefits-and-business-cases/>
- [5] L. C. Dieu, „Closed-Loop vs. Open-Loop Payment Systems: Which One Rules the Future of Transactions?“, SmartDev. Приступљено: 29. Октобар 2024. [На Интернету]. Available at: <https://smartdev.com/closed-loop-vs-open-loop-payment-systems-which-one-rules-the-future-of-transactions/>
- [6] „Closed-Loop vs. Open-Loop Payment Systems | What’s the Difference“. Приступљено: 29. Октобар 2024. [На Интернету]. Available at: <https://www.billfold.tech/blog/closed-loop-vs-open-loop-payment-systems>
- [7] „Apple Pay, PayPal, Purpl: Open or Closed Loop? - Purpl“. Приступљено: 29. Октобар 2024. [На Интернету]. Available at: <https://purplme.com/blog/apple-pay-paypal-purpl-open-or-closed-loop-digital-wallets/>
- [8] „Cineplexx“. Приступљено: 29. Октобар 2024. [На Интернету]. Available at: <https://www.cineplexx.rs/info/bonus-card>
- [9] N. McCarthy, „Starbucks Holds More Cash Than Many Banks [Infographic]“, Forbes. Приступљено: 29. Октобар 2024. [На Интернету]. Available at: <https://www.forbes.com/sites/niallmccarthy/2016/08/01/starbucks-holds-more-cash-than-many-banks-infographic/>
- [10] Z. Miller, „What’s behind the move to closed loop payments and branded digital wallets“, Tearsheet. Приступљено: 29. Октобар 2024. [На Интернету]. Available at: <https://tearsheet.co/payments/whats-behind-the-move-to-closed-loop-payments-and-branded-digital-wallets/>
- [11] „What is open banking?“, Open Banking. Приступљено: 06. Новембар 2024. [На Интернету]. Available at: <https://www.openbanking.org.uk/what-is-open-banking/>
- [12] „Why open banking is safe“, Open Banking. Приступљено: 06. Новембар 2024. [На Интернету]. Available at: <https://www.openbanking.org.uk/why-open-banking-is-safe/>

- [13] „Корисно је да знате... Шта ново доносе измене и допуне Закона о платним услугама“. Приступљено: 06. Новембар 2024. [На Интернету]. Available at: <https://www.nbs.rs/sr/scripts/showcontent/index.html?id=19921>
- [14] „Cyber incidents in the financial industry worldwide 2023“, Statista. Приступљено: 25. Октобар 2024. [На Интернету]. Available at: <https://www.statista.com/statistics/1310985/number-of-cyber-incidents-in-financial-industry-worldwide/>
- [15] „Cost of a data breach in financial sector worldwide 2024“, Statista. Приступљено: 25. Октобар 2024. [На Интернету]. Available at: <https://www.statista.com/statistics/1324063/cost-of-data-breaches-in-financial-industry-worldwide/>
- [16] E. K. Edward Kost, „10 Biggest Data Breaches in Finance“, 10 Biggest Data Breaches in Finance. [На Интернету]. Available at: <https://www.upguard.com/blog/biggest-data-breaches-financial-services>
- [17] „Tokenisation and traditional finance“, Tokenisation and traditional finance. [На Интернету]. Available at: <https://assets.kpmg.com/content/dam/kpmgsites/xx/pdf/2024/04/tokenisation-and-traditional-finance.pdf.coredownload.inline.pdf>
- [18] „World Bank Payment Systems“, World Bank. Приступљено: 30. Октобар 2024. [На Интернету]. Available at: <https://www.worldbank.org/en/topic/paymentsystemsremittances>
- [19] R. Lai, „Chapter 12 - Understanding Interbank Real-Time Retail Payment Systems“, у *Handbook of Blockchain, Digital Finance, and Inclusion, Volume 1*, D. Lee Kuo Chuen и R. Deng, Ур., Academic Press, 2018, стр. 283–310. doi: 10.1016/B978-0-12-810441-5.00012-9.
- [20] V. Marianne, „Retail Payment Systems: What Do We Learn from Two-Sided Markets?“, Феб. 2006, стр. 6–7. doi: 10.2139/ssrn.877866.
- [21] M. A. Hassan и Z. Shukur, „Review of Digital Wallet Requirements“, у *2019 International Conference on Cybersecurity (ICoCSec)*, Сеп. 2019, стр. 43–48. doi: 10.1109/ICoCSec47621.2019.8970996.
- [22] Karnataka, „STUDY ON CONSUMER PERCEPTION TOWARDS DIGITAL WALLET“, Авг. 2024, Приступљено: 31. Октобар 2024. [На Интернету]. Available at: https://ijrar.com/upload_issue/ijrar_issue_20542959.pdf
- [23] D.-N. Le, R. Kumar, B. Mishra, M. Khari, и J. Chatterjee, *Cyber Security in Parallel and Distributed Computing*. 2019. doi: 10.1002/9781119488330.
- [24] O. Ilina, K. Tagirova, A. Barger, и V. Gorgadze, „SoK: How Blockchain and Tokenization Will Transform the Energy Sector“, у *2023 IEEE International Conference on Omni-layer Intelligent Systems (COINS)*, Јули 2023, стр. 1–6. doi: 10.1109/COINS57856.2023.10189196.
- [25] Z. Chen и K. Omote, „Toward Achieving Anonymous NFT Trading“, *IEEE Access*, том 10, стр. 130166–130176, 2022, doi: 10.1109/ACCESS.2022.3228787.
- [26] B. Pillai, K. Biswas, Z. Hóu, и V. Muthukkumarasamy, „Cross-Blockchain Technology: Integration Framework and Security Assumptions“, *IEEE Access*, том 10, стр. 41239–41259, 2022, doi: 10.1109/ACCESS.2022.3167172.
- [27] S. Chevet, „Blockchain Technology and Non-Fungible Tokens: Reshaping Value Chains in Creative Industries“, 10. Maj 2018., *Social Science Research Network, Rochester, NY*: 3212662. doi: 10.2139/ssrn.3212662.
- [28] A. Panwar и V. Bhatnagar, „Distributed Ledger Technology (DLT): The Beginning of a Technological Revolution for Blockchain“, у *2nd International Conference on Data, Engineering and Applications (IDEA)*, Феб. 2020, стр. 1–5. doi: 10.1109/IDEA49133.2020.9170699.
- [29] M. Nofer, P. Gomber, O. Hinz, и D. Schiereck, „Blockchain“, *Bus. Inf. Syst. Eng.*, том 59, изд. 3, стр. 183–187, Јуни 2017, doi: 10.1007/s12599-017-0467-3.
- [30] Z. Zheng, S. Xie, H. Dai, X. Chen, и H. Wang, „An Overview of Blockchain Technology: Architecture, Consensus, and Future Trends“, у *2017 IEEE International Congress on Big Data (BigData Congress)*, Јуни 2017, стр. 557–564. doi: 10.1109/BigDataCongress.2017.85.
- [31] D. Yaga, P. Mell, N. Roby, и K. Scarfone, „Blockchain technology overview“, National Institute of Standards and Technology, Gaithersburg, MD, NIST IR 8202, Окт. 2018. doi: 10.6028/NIST.IR.8202.

- [32] Y. W. Gang Xiong, „Survey of security supervision on blockchain from the perspective of technology“,
- [33] „Blockchain.com | Charts - Estimated Transaction Value (USD)“. Приступљено: 25. Октобар 2024. [На Интернету]. Available at: [https://www.blockchain.com/explorer/charts/\[id\]](https://www.blockchain.com/explorer/charts/[id])
- [34] A. A. of B. in B. Y. L. Application Analysis of Blockchain in Business Yimiao Li, „Application Analysis of Blockchain in Business“,
- [35] A. R. Anton Ruddenklau, „Total fintech investment tops US\$210 billion, as interest in crypto and blockchain surges, says KPMG’s Pulse of Fintech“, Total fintech investment tops US\$210 billion, as interest in crypto and blockchain surges, says KPMG’s Pulse of Fintech. [На Интернету]. Available at: <https://kpmg.com/xx/en/media/press-releases/2022/02/total-fintech-investment-tops-us-210-billion.html>
- [36] D. Vujičić, D. Jagodić, и S. Randić, „Blockchain technology, bitcoin, and Ethereum: A brief overview“, у *2018 17th International Symposium INFOTEH-JAHORINA (INFOTEH)*, Март 2018, стр. 1–6. doi: 10.1109/INFOTEH.2018.8345547.
- [37] A. H. Mohammed, A. A. Abdulateef, и I. A. Abdulateef, „Hyperledger, Ethereum and Blockchain Technology: A Short Overview“, у *2021 3rd International Congress on Human-Computer Interaction, Optimization and Robotic Applications (HORA)*, Јуни 2021, стр. 1–6. doi: 10.1109/HORA52670.2021.9461294.
- [38] J. L. Carter и M. N. Wegman, „Universal classes of hash functions (Extended Abstract)“, у *Proceedings of the ninth annual ACM symposium on Theory of computing*, у STOC ’77. New York, NY, USA: Association for Computing Machinery, 1977, стр. 106–112. doi: 10.1145/800105.803400.
- [39] V. Mandić, B. Мандић, R. Ramač, P. Рамач, A. Katin, и A. Катин, *Uvod u algoritme i strukture podataka: [priručnik za vežbe: odabrani primeri i zadaci]*, том 995. у Edicija „Tehničke nauke - udžbenici“, vol. 995. Novi Sad: Fakultet tehničkih nauka, 2022. Приступљено: 31. Октобар 2024. [На Интернету]. Available at: <https://plus.cobiss.net/cobiss/sr/sr/bib/68240137>
- [40] B. Preneel, „Cryptographic hash functions“, *Eur. Trans. Telecommun.*, том 5, изд. 4, стр. 431–448, 1994, doi: 10.1002/ett.4460050406.
- [41] „SHA-1 Hash“, GeeksforGeeks. Приступљено: 31. Октобар 2024. [На Интернету]. Available at: <https://www.geeksforgeeks.org/sha-1-hash-in-java/>
- [42] M. Szydło, „Merkle Tree Traversal in Log Space and Time“, у *Advances in Cryptology - EUROCRYPT 2004*, C. Cachin и J. L. Camenisch, Ур., Berlin, Heidelberg: Springer, 2004, стр. 541–554. doi: 10.1007/978-3-540-24676-3_32.
- [43] H. Li, R. Lu, L. Zhou, B. Yang, и X. Shen, „An Efficient Merkle-Tree-Based Authentication Scheme for Smart Grid“, *IEEE Syst. J.*, том 8, изд. 2, стр. 655–663, Јуни 2014, doi: 10.1109/JSYST.2013.2271537.
- [44] N. Alshuqayran, N. Ali, и R. Evans, „A Systematic Mapping Study in Microservice Architecture“, у *2016 IEEE 9th International Conference on Service-Oriented Computing and Applications (SOCA)*, Нов. 2016, стр. 44–51. doi: 10.1109/SOCA.2016.15.
- [45] J. Esparza-Pedro, F. D. Muñoz-Escóí, и J. M. Bernabéu-Aubán, „Modeling microservice architectures“, *J. Syst. Softw.*, том 213, стр. 112041, Јули 2024, doi: 10.1016/j.jss.2024.112041.
- [46] T. Cerny, M. J. Donahoo, и M. Trnka, „Contextual understanding of microservice architecture: current and future directions“, *ACM SIGAPP Appl. Comput. Rev.*, том 17, изд. 4, стр. 29–45, Јан. 2018, doi: 10.1145/3183628.3183631.
- [47] Y. Xing и Y. Zhan, „Virtualization and Cloud Computing“, у *Future Wireless Networks and Information Systems*, Y. Zhang, Ур., Berlin, Heidelberg: Springer, 2012, стр. 305–312. doi: 10.1007/978-3-642-27323-0_39.
- [48] L. De Lauretis, „From Monolithic Architecture to Microservices Architecture“, у *2019 IEEE International Symposium on Software Reliability Engineering Workshops (ISSREW)*, Окт. 2019, стр. 93–96. doi: 10.1109/ISSREW.2019.00050.
- [49] L. Abdollahi Vayghan, M. A. Saied, M. Toeroe, и F. Khendek, „Microservice Based Architecture: Towards High-Availability for Stateful Applications with Kubernetes“, у *2019*

IEEE 19th International Conference on Software Quality, Reliability and Security (QRS), Јули 2019, стр. 176–185. doi: 10.1109/QRS.2019.00034.

[50] „Home“, C4 model. Приступљено: 01. Новембар 2024. [На Интернету]. Available at: <https://c4model.com/>

[51] „Factory method Design Pattern“, GeeksforGeeks. Приступљено: 06. Новембар 2024. [На Интернету]. Available at: <https://www.geeksforgeeks.org/factory-method-for-designing-pattern/>

PRILOG A: IZVORNI KOD PROGRAM.CS KLASE API GATEWAY-A

```
using APIGateway.Extensions;
using Yarp.ReverseProxy.Transforms;

var builder = WebApplication.CreateBuilder(args);

// Add services to the container.
builder.Services.AddControllers();
builder.Services.AddEndpointsApiExplorer();

builder.Services
    .AddAuthorization(options =>
    {
        options.AddPolicy("authenticated", policy =>
        {
            policy.RequireAuthenticatedUser();
        });
        options.AddPolicy("user", policy =>
        {
            policy.RequireAuthenticatedUser();
            policy.RequireRole("ROLE_USER");
        });
        options.AddPolicy("admin", policy =>
        {
            policy.RequireAuthenticatedUser();
            policy.RequireRole("ROLE_ADMIN");
        });
        options.AddPolicy("system", policy =>
        {
            policy.RequireAuthenticatedUser();
            policy.RequireRole("ROLE_SYSTEM");
        });
    })
    .AddAuthentication()
    .AddZitadelIntrospection(
        "ZITADEL_BASIC",
        o =>
        {
            o.Authority = builder.Configuration.GetValue<string>("Zitadel:Domain");
            o.ClientId = builder.Configuration.GetValue<string>("Zitadel:ClientId");
            o.ClientSecret = builder.Configuration.GetValue<string>("Zitadel:ClientSecret");
        });

builder.Services.AddReverseProxy()
    .LoadFromConfig(builder.Configuration.GetSection("ReverseProxy"))
    .AddTransforms(builderContext =>
```

```

    {
        if (!string.IsNullOrEmpty(builderContext.Route.AuthorizationPolicy))
        {
            builderContext.AddRequestTransform(async transformContext =>
            {
                var claimSub = transformContext.HttpContext.User.Claims.FirstOrDefault(c => c.Type
                == "sub")?.Value;

                if (!string.IsNullOrEmpty(claimSub))
                {
                    transformContext.ProxyRequest.Headers.Add("sub", claimSub);
                }
            });
        }
    });
});

var app = builder.Build();
app.UseHttpsRedirection();
app.UseAuthentication();
app.UseAuthorization();
app.MapReverseProxy();
app.MapControllers();
app.Run();

```

PRILOG B: IZVORNI KOD APPSETTINGS.JSON KONFIGURACIONOG FAJLA API GATEWAY-A

```

"ReverseProxy": {
  "Routes": {
    "wallet-route": {
      "ClusterId": "wallet-cluster",
      "AuthorizationPolicy": "authenticated",
      "Match": {
        "Path": "wallet/getWalletForCurrentUser",
        "Methods": [ "GET" ]
      }
    },
    "payment-route": {
      "ClusterId": "transaction-cluster",
      "AuthorizationPolicy": "authenticated",
      "Match": {
        "Path": "transaction/payment",
        "Methods": [ "POST" ]
      }
    },
    "deposit-route": {
      "ClusterId": "transaction-cluster",
      "AuthorizationPolicy": "admin",
      "Match": {
        "Path": "transaction/deposit",
        "Methods": [ "POST" ]
      }
    },
    "withdraw-route": {
      "ClusterId": "transaction-cluster",
      "AuthorizationPolicy": "admin",
      "Match": {
        "Path": "transaction/withdraw",
        "Methods": [ "POST" ]
      }
    }
  },
  "Clusters": {
    "wallet-cluster": {
      "Destinations": {
        "destination1": {
          "Address": "https://localhost:7187/api/"
        }
      }
    },
    "transaction-cluster": {
      "Destinations": {
        "destination1": {

```



```

        "Address": "https://localhost:7036/api/"
    }
}
}
}
}

```

PRILOG C: IZVORNI KOD WALLET KONTROLERA WALLET MIKROSERVISA

```

using Microsoft.AspNetCore.Mvc;
using WalletApi.Dtos.Enums;
using WalletApi.Dtos.TokenDtos;
using WalletApi.Dtos.WalletDtos;
using WalletApi.Exceptions.CustomExceptions;
using WalletApi.Exceptions.ExceptionErrorCodes;
using WalletApi.Services;

namespace WalletApi.Controllers
{
    [Route("api/[controller]")]
    [ApiController]
    public class WalletController : ControllerBase
    {
        private readonly IWalletService WalletService;
        private readonly ITokenService TokenService;
        private readonly IRedisService RedisService;
        private const string LockedWalletsKey = "LockedWallets";

        public WalletController(IWalletService walletService, ITokenService tokenService,
            IRedisService redisService)
        {
            WalletService = walletService;
            TokenService = tokenService;
            RedisService = redisService;
        }

        [HttpGet("getWalletForCurrentUser")]
        public async Task<ActionResult<WalletDto>> GetWallet()
        {
            var sub = HttpContext.Request.Headers["sub"].FirstOrDefault();

            if (String.IsNullOrEmpty(sub))
            {
                throw new
                BadRequestException(BadRequestExceptionErrorCodes.SubKeyNotProvidedInHeader, "There is no sub key
                provided in requests header.");
            }

            var wallet = await WalletService.GetWallet(sub);

            if (wallet is null)
            {
                wallet = await WalletService.CreateWallet(sub);
            }

            return Ok(wallet);
        }

        [HttpGet("getTokenForTransaction/{tokenType}/{sub}")]
        public async Task<ActionResult<UpdateTokenDto>> GetTokenForTransaction([FromRoute] string
            sub, [FromRoute] TokenTypeEnum tokenType)
        {
            var wallet = await WalletService.GetWallet(sub);
            if (wallet is null)
            {
                throw new NotFoundException(NotFoundExceptionErrorCodes.WalletNotFound, "User that
                wants to withdraw or deposit does not have created wallet.");
            }

            var lockedWallets = await RedisService.GetListAsync<string>(LockedWalletsKey);
            if (lockedWallets.Contains(wallet.Address.ToString()))
            {
                throw new ForbiddenException(ForbiddenExceptionErrorCodes.LockedWallet, "User wallet
                is locked by another transaction.");
            }
        }
    }
}

```

```

        var tokenForTransaction = await TokenService.GetTokenForTransaction(wallet.Address,
tokenType);

        lockedWallets.Add(wallet.Address.ToString());
        await RedisService.StoreListAsync<string>(LockedWalletsKey, lockedWallets);

        return Ok(tokenForTransaction);
    }

    [HttpGet("getTokensForTransaction/{tokenType}/{from}/{to}")]
    public async Task<ActionResult<TokensForTransactionDto>> GetTokensForTransaction([FromRoute]
string from, [FromRoute] string to, [FromRoute] TokenTypeEnum tokenType)
    {
        (Guid walletIdFrom, Guid walletIdTo) = await
WalletService.GetWalletsForTransaction(from, to);

        var lockedWallets = await RedisService.GetListAsync<string>(LockedWalletsKey);
        if (lockedWallets.Any(w => w == walletIdFrom.ToString() || w == walletIdTo.ToString()))
        {
            throw new ForbiddenException(ForbiddenExceptionErrorCodes.LockedWallet, "Some of
wallets are locked by another transaction.");
        }

        var tokensForTransaction = await TokenService.GetTokensForTransaction(walletIdFrom,
walletIdTo, tokenType);

        lockedWallets.Add(walletIdTo.ToString());
        lockedWallets.Add(walletIdFrom.ToString());
        await RedisService.StoreListAsync<string>(LockedWalletsKey, lockedWallets);

        return Ok(tokensForTransaction);
    }

    [HttpPost("updateTokensByTransaction")]
    public async Task<ActionResult> UpdateTokensByTransaction([FromBody] TokensForTransactionDto
updatedTokens)
    {
        await TokenService.UpdateTokensByTransaction(updatedTokens.TokenFrom,
updatedTokens.TokenTo);
        var lockedWallets = await RedisService.GetListAsync<string>(LockedWalletsKey);
        if (updatedTokens.TokenFrom is not null)
        {
            lockedWallets.Remove(updatedTokens.TokenFrom.WalletAddress.ToString());
        }
        if (updatedTokens.TokenTo is not null)
        {
            lockedWallets.Remove(updatedTokens.TokenTo.WalletAddress.ToString());
        }

        await RedisService.StoreListAsync(LockedWalletsKey, lockedWallets);
        return Ok();
    }
}
}

```

PRILOG D: IZVORNI KOD WALLET SERVISNE KLASSE WALLET MIKROSERVISA

```

using WalletEFDAL.Models;
using WalletEFDAL.DataAccess;
using Microsoft.EntityFrameworkCore;
using WalletApi.Dtos.WalletDtos;
using AutoMapper;
using WalletApi.Exceptions.CustomExceptions;
using WalletApi.Exceptions.ExceptionErrorCodes;
using Newtonsoft.Json;
using System.Dynamic;
using WalletApi.Dtos.Enums;

namespace WalletApi.Services.Impl
{
    public class WalletService : IWalletService
    {
        private readonly WalletDbContext Context;
        private readonly IAzureKeyVaultService AzureKeyVaultService;
    }
}

```

```

        private readonly IMapper Mapper;

        public WalletService(WalletDbContext context, IAzureKeyVaultService azureKeyVaultService,
IMapper mapper)
        {
            Context = context;
            AzureKeyVaultService = azureKeyVaultService;
            Mapper = mapper;
        }

        public async Task<WalletDto> GetWallet(string sub)
        {
            var walletIdFromSecret = await AzureKeyVaultService.GetSecret(sub);

            Guid.TryParse(walletIdFromSecret, out Guid walletId);

            var wallet = await Context.Wallets
                .Include(w => w.Tokens)
                .FirstOrDefaultAsync(it => it.Address == walletId);

            if (wallet is null)
            {
                return null;
            }

            WalletDto walletDto = new WalletDto() { Address = wallet.Address };
            foreach (var token in wallet.Tokens)
            {
                dynamic tokenData = JsonConvert.DeserializeObject<ExpandoObject>(token.TokenData);
                walletDto.Tokens.Add(new Dtos.TokenDtos.TokenDto()
                {
                    TokenId = token.TokenId,
                    Balance = (decimal)tokenData.Balance,
                    TokenType = (TokenTypeEnum)tokenData.TokenTypeId,
                });
            }

            return walletDto;
        }

        public async Task<(Guid walletAddressFrom, Guid WalletAddressTo)>
GetWalletsForTransaction(string subFrom, string subTo)
        {
            var walletFrom = await GetWallet(subFrom);
            if (walletFrom is null)
            {
                throw new NotFoundException(NotFoundExceptionErrorCodes.WalletNotFound, "User that
wants to pay does not have created wallet.");
            }

            var walletTo = await GetWallet(subTo);
            if (walletTo is null)
            {
                throw new NotFoundException(NotFoundExceptionErrorCodes.WalletNotFound, "User to be
paid does not have created wallet.");
            }

            return (walletFrom.Address, walletTo.Address);
        }

        public async Task<WalletDto> CreateWallet(string sub)
        {
            var walletId = Guid.NewGuid();
            await AzureKeyVaultService.CreateSecret(sub, walletId.ToString());

            var wallet = Context.Wallets.Add(new Wallet { Address = walletId });
            await Context.SaveChangesAsync();

            return Mapper.Map<WalletDto>(wallet.Entity); ;
        }
    }
}

```

PRILOG E: IZVORNI KOD TOKEN SERVISNE KLASSE WALLET MIKROSERVISA

```
using AutoMapper;
using Microsoft.EntityFrameworkCore;
using System.Net;
using WalletApi.Dtos.Enums;
using WalletApi.Dtos.TokenDtos;
using WalletEFDAL.DataAccess;
using WalletEFDAL.Models;

namespace WalletApi.Services.Impl
{
    public class TokenService : ITokenService
    {
        private readonly WalletDbContext Context;
        private readonly IMapper Mapper;

        public TokenService(WalletDbContext context, IMapper mapper)
        {
            Context = context;
            Mapper = mapper;
        }

        public async Task<UpdateTokenDto> GetTokenForTransaction(Guid address, TokenTypeEnum
tokenType)
        {
            var token = await Context.Tokens
                .FirstOrDefaultAsync(t => t.WalletAddress == address &&
EF.Functions.Like(t.TokenData, "%\"TokenTypeId\":\" + (int)tokenType + \"%"));

            return token is null ? new UpdateTokenDto() { WalletAddress = address } :
Mapper.Map<UpdateTokenDto>(token);
        }

        public async Task<TokensForTransactionDto> GetTokensForTransaction(Guid fromAddress, Guid
toAddress, TokenTypeEnum tokenType)
        {
            var fromToken = await Context.Tokens
                .FirstOrDefaultAsync(t => t.WalletAddress == fromAddress &&
EF.Functions.Like(t.TokenData, "%\"TokenTypeId\":\" + (int)tokenType + \"%"));

            var toToken = await Context.Tokens
                .FirstOrDefaultAsync(t => t.WalletAddress == toAddress &&
EF.Functions.Like(t.TokenData, "%\"TokenTypeId\":\" + (int)tokenType + \"%"));

            return new TokensForTransactionDto(
                fromToken is null ? new UpdateTokenDto() { WalletAddress = fromAddress } :
Mapper.Map<UpdateTokenDto>(fromToken),
                toToken is null ? new UpdateTokenDto() { WalletAddress = toAddress } :
Mapper.Map<UpdateTokenDto>(toToken));
        }

        public async Task UpdateTokensByTransaction(UpdateTokenDto fromTokenUpdated, UpdateTokenDto
toTokenUpdated)
        {
            if (fromTokenUpdated is not null)
            {
                var fromToken = await Context.Tokens.FirstOrDefaultAsync(t => t.TokenId ==
fromTokenUpdated.TokenId);
                if (fromToken is null)
                {
                    Context.Add(Mapper.Map<Token>(fromTokenUpdated));
                }
                else
                {
                    UpdateToken(fromToken, fromTokenUpdated);
                }
            }

            if (toTokenUpdated is not null)
            {
                var toToken = await Context.Tokens.FirstOrDefaultAsync(t => t.TokenId ==
toTokenUpdated.TokenId);
                if (toToken is null)
                {
                    Context.Add(Mapper.Map<Token>(toTokenUpdated));
                }
            }
        }
    }
}
```

```

        else
        {
            UpdateToken(toToken, toTokenUpdated);
        }
    }

    await Context.SaveChangesAsync();
}

private void UpdateToken(Token token, UpdateTokenDto tokenUpdated)
{
    token.TokenData = tokenUpdated.TokenData;
    token.TokenDataSignature = tokenUpdated.TokenDataSignature;
}
}
}

```

PRILOG F: IZVORNI KOD REDIS SERVISNE KLASSE WALLET MIKROSERVISA

```

using Newtonsoft.Json;
using Polly;
using Polly.Retry;
using StackExchange.Redis;

namespace WalletApi.Services.Impl
{
    public class RedisService : IRedisService
    {
        private readonly IDatabase Redis;
        private readonly AsyncRetryPolicy RetryPolicy;
        private readonly ILogger<RedisService> Logger;

        public RedisService(IConnectionMultiplexer redis, ILogger<RedisService> logger)
        {
            Redis = redis.GetDatabase();
            Logger = logger;
            // Polly Retry Logic:
            // This retry policy is designed to handle Redis downtimes gracefully for up to 45
seconds.
            // The policy will react to RedisConnectionExceptions using an exponential backoff
strategy:
            // 1st retry: After 3 seconds.
            // 2nd retry: After 6 seconds.
            // 3rd retry: After 12 seconds.
            // 4th retry: After 24 seconds.
            // This gives a total retry window of 45 seconds (3 + 6 + 12 + 24).
            // It complements the StackExchange.Redis connection settings which have their own retry
mechanisms,
            // ensuring robustness in the face of transient Redis outages.
            RetryPolicy = Policy
                .Handle<RedisConnectionException>()
                .WaitAndRetryAsync(4, // retry 4 times
                    retryAttempt => TimeSpan.FromSeconds(Math.Pow(2, retryAttempt + 1)), // exponential
backoff
                    (exception, timeSpan, retryCount, context) =>
                    {
                        Logger.LogWarning("Redis operation failed with message: {message}. Waiting
{timeSpan} before next retry. Retry attempt {retryCount}.", exception.Message, timeSpan,
retryCount);
                    });
        }

        public async Task<List<T>> GetListAsync<T>(string key)
        {
            RedisValue value = await GetRedisValueAsync(key);

            return value.IsNullOrEmpty ? new List<T>() :
JsonConvert.DeserializeObject<List<T>>(value.ToString());
        }

        public async Task StoreListAsync<T>(string key, List<T> value, TimeSpan? expiry = null)
        {
            await Redis.StringSetAsync(key, JsonConvert.SerializeObject(value), expiry);
        }
    }
}

```

```

        private async Task<RedisValue> GetRedisValueAsync(string key)
        {
            return await RetryPolicy.ExecuteAsync(() => Redis.StringGetAsync(key));
        }
    }
}

```

PRILOG G: IZVORNI KOD AZURE KEY VAULT SERVISNE KLASSE WALLET MIKROSERVISA

```

using Azure.Security.KeyVault.Secrets;

namespace WalletApi.Services.Impl
{
    public class AzureKeyVaultService : IAzureKeyVaultService
    {
        private readonly SecretClient SecretClient;
        public AzureKeyVaultService(SecretClient secretClient)
        {
            SecretClient = secretClient;
        }

        public async Task<string> GetSecret(string key)
        {
            try
            {
                var secret = (await SecretClient.GetSecretAsync(key)).Value.Value;
                return secret;
            }
            catch (Azure.RequestFailedException)
            {
                return null;
            }
        }

        public async Task CreateSecret(string key, string value)
        {
            await SecretClient.SetSecretAsync(new KeyVaultSecret(key, value));
        }
    }
}

```

PRILOG H: IZVORNI KOD TOKEN KONTROLERA TOKEN MIKROSERVISA

```

using Microsoft.AspNetCore.Mvc;
using TokenApi.Dtos.Enums;
using TokenApi.Dtos.TokenDtos;
using TokenApi.Services;

namespace TokenApi.Controllers
{
    [Route("api/[controller]")]
    [ApiController]
    public class TokenController : ControllerBase
    {
        private readonly ITokenService TokenService;

        public TokenController(ITokenService tokenService)
        {
            TokenService = tokenService;
        }

        [HttpPost("createNewToken/{walletAddress}/{transactionId}/{transactionType}/{tokenType}/{amount}")]
        public async Task<ActionResult<TokenDto>> CreateNewToken([FromRoute] Guid walletAddress,
            [FromRoute] Guid transactionId, [FromRoute] TransactionTypeEnum transactionType, [FromRoute]
            TokenTypeEnum tokenType, [FromRoute] decimal amount, [FromBody] TokenDto currentToken)
        {

```

```

        if (transactionType != TransactionTypeEnum.Deposit && transactionType !=
TransactionTypeEnum.Withdraw)
        {
            return BadRequest("Wrong transaction type.");
        }
        var newToken = TokenService.CreateNewToken(walletAddress, transactionId,
transactionType, tokenType, amount, currentToken, null);
        return Ok(newToken);
    }

    [HttpPost("createNewTokens/{transactionId}/{transactionType}/{tokenType}/{amount}")]
    public async Task<ActionResult<TokensForTransactionDto>> CreateNewTokens([FromRoute] Guid
transactionId, [FromRoute] TransactionTypeEnum transactionType, [FromRoute] TokenTypeEnum tokenType,
[FromRoute] decimal amount, [FromBody] TokensForTransactionDto tokensForTransaction)
    {
        if (transactionType != TransactionTypeEnum.Payment)
        {
            return BadRequest("Wrong transaction type.");
        }
        var newTokenFrom =
TokenService.CreateNewToken(tokensForTransaction.TokenFrom.WalletAddress, transactionId,
transactionType, tokenType, amount, tokensForTransaction.TokenFrom, true);
        var newTokenTo = TokenService.CreateNewToken(tokensForTransaction.TokenTo.WalletAddress,
transactionId, transactionType, tokenType, amount, tokensForTransaction.TokenTo, false);
        var newTokens = new TokensForTransactionDto
        {
            TokenFrom = newTokenFrom,
            TokenTo = newTokenTo
        };
        return Ok(newTokens);
    }

    [HttpPost("validateToken")]
    public ActionResult<bool> ValidateToken(TokenDto token)
    {
        var result = TokenService.ValidateToken(token);
        return Ok(result);
    }
}
}

```

PRILOG I: IZVORNI KOD TOKEN SERVISNE KLASSE TOKEN MIKROSERVISA

```

using Newtonsoft.Json;
using Newtonsoft.Json.Linq;
using TokenApi.Dtos.Enums;
using TokenApi.Dtos.TokenDtos;
using TokenApi.Models.TokenVersions;

namespace TokenApi.Services.Impl
{
    public class TokenService : ITokenService
    {
        public TokenService() { }

        public TokenDto CreateNewToken(Guid walletAddress, Guid TransactionId, TransactionTypeEnum
transactionType, TokenTypeEnum tokenType, decimal amount, TokenDto currentToken, bool? isFrom)
        {
            var tokenFactory = new TokenFactory();
            if (currentToken.TokenId == Guid.Empty)
            {
                return tokenFactory.CreateToken(walletAddress, amount, TransactionId,
transactionType, tokenType).ToTokenDto();
            }

            var isValid = ValidateToken(currentToken);
            if (!isValid)
            {
                throw new Exception("Invalid token with Id: " + currentToken.TokenId);
            }

            return tokenFactory.UpdateToken(currentToken, transactionType, amount,
isFrom).ToTokenDto();
        }
    }
}

```

```

    }

    public bool ValidateToken(TokenDto currentToken)
    {
        var tokenVersion =
JsonConvert.DeserializeObject<JObject>(currentToken.TokenData) ["Version"].ToString();
        var tokenFactory = new TokenFactory();
        IToken token = tokenFactory.GetToken(tokenVersion, currentToken);
        return token.IsTokenValid();
    }
}
}

```

PRILOG J: IZVORNI KOD TRANSACTION KONTROLERA TRANSACTION MIKROSERVISA

```

using Microsoft.AspNetCore.Mvc;
using TransactionApi.Dtos.TokenDtos;
using TransactionApi.Dtos.TransactionDtos;
using TransactionApi.Services;
using TransactionEFDAL.Models.Enums;

namespace TransactionApi.Controllers
{
    [Route("api/[controller]")]
    [ApiController]
    public class TransactionController : ControllerBase
    {
        private readonly ITransactionService TransactionService;
        public TransactionController(ITransactionService transactionService)
        {
            TransactionService = transactionService;
        }

        [HttpPost("payment")]
        public async Task<ActionResult> CreateTransactionPayment([FromBody]
PaymentTransactionCreateDto transactionCreateDto)
        {
            if (transactionCreateDto.TransactionType != TransactionTypeEnum.Payment) {
                return BadRequest("Wrong transaction type.");
            }

            Guid transactionId = Guid.NewGuid();
            var currentTokens = await
TransactionService.GetTokensForTransaction(transactionCreateDto.TokenType,
transactionCreateDto.From, transactionCreateDto.To);
            var createdTokens = await TransactionService.CreateNewTokens(transactionId,
currentTokens, transactionCreateDto.Amount, transactionCreateDto.TokenType);
            var transactionToProcess = TransactionService.CreateTransactionToProcess(transactionId,
TransactionStatus.Success, transactionCreateDto.From, currentTokens, createdTokens,
transactionCreateDto.Amount, transactionCreateDto.TokenType);
            TransactionService.AddTransactionToPool(transactionToProcess);

            return Ok();
        }

        [HttpPost("deposit")]
        public async Task<ActionResult> CreateTransactionDeposit([FromBody]
DepositOrWithdrawTransactionCreateDto transactionCreateDto)
        {
            if (transactionCreateDto.TransactionType != TransactionTypeEnum.Deposit)
            {
                return BadRequest("Wrong transaction type.");
            }

            Guid transactionId = Guid.NewGuid();
            var currentToken = await
TransactionService.GetTokenForTransaction(transactionCreateDto.TokenType,
transactionCreateDto.Address);
            var createdToken = await TransactionService.CreateNewToken(currentToken.WalletAddress,
transactionId, TransactionTypeEnum.Deposit, currentToken, transactionCreateDto.Amount,
transactionCreateDto.TokenType);
            var transactionToProcess = TransactionService.CreateTransactionToProcess(transactionId,
TransactionStatus.Success,
transactionCreateDto.Address,

```



```

        new TokensForTransactionDto() { TokenFrom = null, TokenTo = currentToken },
        new TokensForTransactionDto() { TokenFrom = null, TokenTo = createdToken },
        transactionCreateDto.Amount, transactionCreateDto.TokenType);
TransactionService.AddTransactionToPool(transactionToProcess);

    return Ok();
}

[HttpPost("withdraw")]
public async Task<ActionResult> CreateTransactionWithdraw([FromBody]
DepositOrWithdrawTransactionCreateDto transactionCreateDto)
{
    if (transactionCreateDto.TransactionType != TransactionTypeEnum.Withdraw)
    {
        return BadRequest("Wrong transaction type.");
    }

    Guid transactionId = Guid.NewGuid();
    var currentToken = await
TransactionService.GetTokenForTransaction(transactionCreateDto.TokenType,
transactionCreateDto.Address);
    var createdToken = await TransactionService.CreateNewToken(currentToken.WalletAddress,
transactionId, TransactionTypeEnum.Withdraw, currentToken, transactionCreateDto.Amount,
transactionCreateDto.TokenType);
    var transactionToProcess = TransactionService.CreateTransactionToProcess(
        transactionId,
        TransactionStatus.Success,
        transactionCreateDto.Address,
        new TokensForTransactionDto() { TokenFrom = currentToken, TokenTo = null },
        new TokensForTransactionDto() { TokenFrom = createdToken, TokenTo = null },
        transactionCreateDto.Amount, transactionCreateDto.TokenType);
    TransactionService.AddTransactionToPool(transactionToProcess);

    return Ok();
}
}
}

```

PRILOG K: IZVORNI KOD TRANSACTION SERVISNE KLASSE TRANSACTION MIKROSERVISA

```

using TransactionApi.HostedServices;
using TransactionEFDAL.DataAccess;
using TransactionEFDAL.Models;
using Newtonsoft.Json;
using TransactionApi.Dtos.TokenDtos;
using TransactionEFDAL.Models.Enums;
using System.Net.Http.Headers;
using TransactionApi.Models;
using TransactionApi.Utils;
using System.Net;

namespace TransactionApi.Services.Impl
{
    public class TransactionService : ITransactionService
    {
        private readonly TransactionDbContext Context;
        private readonly TransactionPoolHostedService TransactionPoolHostedService;
        private readonly IHttpClientFactory HttpClientFactory;

        public TransactionService(TransactionPoolHostedService transactionPoolHostedService,
TransactionDbContext context, IHttpClientFactory httpClientFactory)
        {
            Context = context;
            TransactionPoolHostedService = transactionPoolHostedService;
            HttpClientFactory = httpClientFactory;
        }

        public void AddTransactionToPool(TransactionToProcess transactionToProcess)
        {
            TransactionPoolHostedService.AddTransaction(transactionToProcess);
        }

        public TransactionToProcess CreateTransactionToProcess(Guid transactionId, TransactionStatus
status, string iniciator, TokensForTransactionDto currentTokens, TokensForTransactionDto newTokens,
decimal amount, TokenTypeEnum tokenType)

```

```

{
    TransactionToProcess transactionToProcess = new();
    transactionToProcess.Transaction = new Transaction()
    {
        TransactionId = transactionId,
        TransactionType = TransactionTypeEnum.Payment,
        TokenFromOld = JsonConvert.SerializeObject(currentTokens.TokenFrom),
        TokenToOld = JsonConvert.SerializeObject(currentTokens.TokenTo),
        TokenType = tokenType,
        Amount = amount,
        FeeFrom = 0,
        FeeTo = 0,
        Created = DateTime.UtcNow,
        TransactionStatus = status,
        TransactionInitiator = initiator
    };

    transactionToProcess.Transaction.TransactionSignature =
HashFunctions.ComputeSha256Hash(JsonConvert.SerializeObject(transactionToProcess.Transaction));
    transactionToProcess.CreatedTokens = newTokens;

    return transactionToProcess;
}

public async Task<TokenDto> CreateNewToken(Guid walletAddress, Guid transactionId,
TransactionTypeEnum transactionType, TokenDto currentToken, decimal amount, TokenTypeEnum tokenType)
{
    var client = HttpClientFactory.CreateClient("TokenApi");

    var mediaType = new MediaTypeHeaderValue("application/json");
    StringContent jsonContent = new(JsonConvert.SerializeObject(currentToken), mediaType);

    var response = await client.PostAsync("/api/token/createNewToken/" +
walletAddress.ToString() + "/" +
transactionId.ToString() + "/" +
transactionType + "/" +
tokenType + "/" +
amount, jsonContent);

    var newTokens = JsonConvert.DeserializeObject<TokenDto>(await
response.Content.ReadAsStringAsync());

    return newTokens;
}

public async Task<TokensForTransactionDto> CreateNewTokens(Guid transactionId,
TokensForTransactionDto currentTokens, decimal amount, TokenTypeEnum tokenType)
{
    var client = HttpClientFactory.CreateClient("TokenApi");

    var mediaType = new MediaTypeHeaderValue("application/json");
    StringContent jsonContent = new (JsonConvert.SerializeObject(currentTokens), mediaType);

    var response = await client.PostAsync("/api/token/createNewTokens/" +
transactionId.ToString() + "/" +
TransactionTypeEnum.Payment.ToString() + "/" +
tokenType + "/" +
amount, jsonContent);

    var newTokens = JsonConvert.DeserializeObject<TokensForTransactionDto>(await
response.Content.ReadAsStringAsync());

    return newTokens;
}

public async Task<TokensForTransactionDto> GetTokensForTransaction(TokenTypeEnum tokenType,
string from, string to)
{
    var client = HttpClientFactory.CreateClient("WalletApi");

    var response = await client.GetAsync("/api/wallet/getTokensForTransaction/" +
tokenType + "/" +
from + "/" +
to);

    var tokensForTransaction = JsonConvert.DeserializeObject<TokensForTransactionDto>(await
response.Content.ReadAsStringAsync());

    return tokensForTransaction;
}

```

```

public async Task<TokenDto> GetTokenForTransaction(TokenTypeEnum tokenType, string address)
{
    var client = HttpClientFactory.CreateClient("WalletApi");

    var response = await client.GetAsync("/api/wallet/getTokenForTransaction/" +
        tokenType + "/" +
        address);

    if (response.StatusCode != HttpStatusCode.OK)
    {
        throw new Exception();
    }

    var token = JsonConvert.DeserializeObject<TokenDto>(await
response.Content.ReadAsStringAsync());

    return token;
}
}
}

```

PRILOG L: IZVORNI KOD TRANSACTIONPOOL HOSTOVANE SERVISNE KLASSE TRANSACTION MIKROSERVISA

```

using System.Collections.Concurrent;
using TransactionEFDAL.DataAccess;
using TransactionApi.Utils;
using TransactionApi.Models;
using Newtonsoft.Json;
using System.Net.Http.Headers;
using System.Dynamic;

namespace TransactionApi.HostedServices
{
    public class TransactionPoolHostedService : IHostedService, IDisposable
    {
        private readonly TransactionDbContext Context;
        private readonly IHttpClientFactory HttpClientFactory;
        private static readonly ConcurrentBag<TransactionToProcess> TransactionPool = new
ConcurrentBag<TransactionToProcess>();
        private static Timer Timer;

        public TransactionPoolHostedService(TransactionDbContext context, IHttpClientFactory
httpClientFactory)
        {
            Context = context;
            HttpClientFactory = httpClientFactory;
        }

        public Task StartAsync(CancellationToken cancellationToken)
        {
            Timer = new Timer(ProcessTransactions, null, TimeSpan.Zero, TimeSpan.FromSeconds(10));
            return Task.CompletedTask;
        }

        public Task StopAsync(CancellationToken cancellationToken)
        {
            Timer?.Change(Timeout.Infinite, 0);
            return Task.CompletedTask;
        }

        public void AddTransaction(TransactionToProcess transactionToProcess)
        {
            TransactionPool.Add(transactionToProcess);
        }

        private async void ProcessTransactions(object state)
        {
            var transactionsToProcess = new List<TransactionToProcess>();

            while (TransactionPool.TryTake(out var transaction))
            {

```

```

        transactionsToProcess.Add(transaction);
    }

    if (transactionsToProcess.Count > 0)
    {
        var transactionsToGenerateMerkleRoot = transactionsToProcess.Select(t =>
t.Transaction.TransactionSignature).ToList();
        var merkleRoot = MerkleRoot.GenerateMerkleRoot(transactionsToGenerateMerkleRoot);
        var clientBlockchain = HttpClientFactory.CreateClient("BlockchainApi");
        var mediaTypeBlockchain = new MediaTypeHeaderValue("application/json");
        StringContent contentBlockchain = new
StringContent(JsonConvert.SerializeObject(merkleRoot), mediaTypeBlockchain);
        var blockchainProofResponse = await
clientBlockchain.PostAsync("/api/blockchain/writeToBlockchain", contentBlockchain);
        var blockchainProof = await blockchainProofResponse.Content.ReadAsStringAsync();

        foreach (var transactionToProcess in transactionsToProcess)
        {
            //create proof
            var proof = MerkleRoot.GetMerklePath(transactionsToGenerateMerkleRoot,
transactionToProcess.Transaction.TransactionSignature, merkleRoot);

            //add proof for new token from
            if (transactionToProcess.CreatedTokens.TokenFrom is not null)
            {
                dynamic tokenDataFrom =
JsonConvert.DeserializeObject<ExpandoObject>(transactionToProcess.CreatedTokens.TokenFrom.TokenData)
;
                tokenDataFrom.TokenProof = JsonConvert.SerializeObject(proof);
                transactionToProcess.CreatedTokens.TokenFrom.TokenData =
JsonConvert.SerializeObject(tokenDataFrom);
            }

            //add proof for new token to
            if (transactionToProcess.CreatedTokens.TokenTo is not null)
            {
                dynamic tokenDataTo =
JsonConvert.DeserializeObject<ExpandoObject>(transactionToProcess.CreatedTokens.TokenTo.TokenData);
                tokenDataTo.TokenProof = JsonConvert.SerializeObject(proof);
                transactionToProcess.CreatedTokens.TokenTo.TokenData =
JsonConvert.SerializeObject(tokenDataTo);
            }

            var client = HttpClientFactory.CreateClient("WalletApi");
            var mediaType = new MediaTypeHeaderValue("application/json");
            StringContent content = new
StringContent(JsonConvert.SerializeObject(transactionToProcess.CreatedTokens), mediaType);

            var response = await client.PostAsync("/api/wallet/updateTokensByTransaction",
content);

            transactionToProcess.Transaction.EtheriumProof = blockchainProof;
            await Context.Transactions.AddAsync(transactionToProcess.Transaction);
        }

        await Context.SaveChangesAsync();
    }
}

public void Dispose()
{
    Timer?.Dispose();
}
}
}

```

PRILOG M: IZVORNI KOD BLOCKCHAIN KONTROLERA BLOCKCHAIN MIKROSERVISA

```

using BlockchainAPI.Services;
using Microsoft.AspNetCore.Mvc;

namespace BlockchainAPI.Controllers
{
    [Route("api/[controller]")]
    [ApiController]
    public class BlockchainController : ControllerBase
    {

```

```

{
    private readonly IBlockchainService BlockchainService;

    public BlockchainController(IBlockchainService blockchainService)
    {
        BlockchainService = blockchainService;
    }

    [HttpPost("writeToBlockchain")]
    public async Task<ActionResult<string>> WriteToBlockchain([FromBody] string merkleRoot)
    {
        var result = await BlockchainService.WriteToBlockchain(merkleRoot);
        return Ok(result);
    }
}
}

```

PRILOG N: IZVORNI KOD ETHEREUM SERVISNE KLASE BLOCKCHAIN MIKROSERVISA

```

using Nethereum.Hex.HexTypes;
using Nethereum.Web3;

namespace BlockchainAPI.Services.Impl
{
    public class EthereumService : IBlockchainService
    {
        private readonly IConfiguration Configuration;
        public EthereumService(IConfiguration configuration)
        {
            Configuration = configuration;
        }

        public async Task<string> WriteToBlockchain(string merkleRoot)
        {
            var url = Configuration.GetValue<string>("EthereumConnection:Url");
            var privateKey = Configuration.GetValue<string>("EthereumConnection:PrivateKey");
            var account = new Nethereum.Web3.Accounts.Account(privateKey);
            var web3 = new Web3(account, url);
            var contractAddress =
                Configuration.GetValue<string>("EthereumConnection:ContractAddress");
            var abi = @"[
            {
                ""inputs"": [{
                    ""internalType"": ""string"",
                    ""name"": ""x"",
                    ""type"": ""string""}],
                ""name"": ""set"",
                ""outputs"": [],
                ""stateMutability"": ""nonpayable"",
                ""type"": ""function""},
            {
                ""inputs"": [],
                ""name"": ""get"",
                ""outputs"": [{
                    ""internalType"": ""string"",
                    ""name"": "",
                    ""type"": ""string""}],
                ""stateMutability"": ""view"",
                ""type"": ""function""},
            {
                ""inputs"": [],
                ""name"": ""storedData"",
                ""outputs"": [{
                    ""internalType"": ""string"",
                    ""name"": "",
                    ""type"": ""string""}],
                ""stateMutability"": ""view"",
                ""type"": ""function""}]]";
            var contract = web3.Eth.GetContract(abi, contractAddress);
            var setFunction = contract.GetFunction("set");
            var transactionHash = await setFunction.SendTransactionAsync(account.Address,
                new HexBigInteger(300000), new HexBigInteger(0), merkleRoot);
            return transactionHash;
        }
    }
}

```

BIOGRAFIJA

Đorđe Šević je rođen 2. maja 1998. godine u Novom Sadu. Osnovnu školu „Jovan Jovanović Zmaj“ u Sremskoj Kamenici završava 2013. godine kao nosilac Vukove diplome. Zatim upisuje Medicinsku školu „7. april“ u Novom Sadu i 2017. godine završava smer Fizioteraputski tehničar. Iste godine upisuje Fakultet tehničkih nauka u Novom Sadu, studijski program Primenjeno softversko inženjerstvo sa kojeg ipak odlučuje da se ispiše 2019. godine i upiše studijski program Inženjerstvo informacionih sistema takođe na Fakultetu tehničkih nauka u Novom Sadu. Sve ispite u okviru osnovnih akademskih studija završava 2024. godine sa prosečnom ocenom 8.87.

Od 2021. godine svoju profesionalnu karijeru započinje kao stipendista i student ambasador u kompaniji TIAC u Novom Sadu gde se pored *part-time* rada na razvoju različitih projekata, bavi i promocijom kompanije u društvu studenata kao i organizacijom praksi i raznih događaja u kompaniji.